DE-VJEPA: Decision-Equivalent Latent Prediction for Policy Fidelity and Control

Anonymous authors
Paper under double-blind review

Abstract

Accurate latent predictions need not preserve a controller's decisions, and preserving those decisions need not improve task success. We study these relationships through DE-VJEPA, which augments deterministic latent regression with a Kullback–Leibler (KL) penalty between a frozen reference policy's action distributions at observed and predicted future representations. The reported comparisons use a frozen ResNet-18 encoder on five MetaWorld tasks and PushT, with evaluation policies trained using different seeds. On clean MetaWorld, the policy-regularized predictor has 15.4% higher latent mean squared error but 19.8% lower decision KL than the latent-only baseline, reversing the model ranking. Decision KL improves in seven of eight dataset–condition comparisons, and its advantage persists through five-step prediction under recorded actions. Reference-policy and action-mean ablations probe the supervision signal; persistence remains a stronger baseline on clean PushT. Across 14,850 closed-loop MetaWorld episodes, however, the tested planners do not establish an aggregate success advantage over latent-only prediction. These findings distinguish latent accuracy, policy fidelity, and control performance as separate evaluation criteria and show the limits of using offline policy agreement to infer control benefits.

1 Introduction

A controller must choose actions whose consequences it has not yet observed. A learned world model can support this choice by predicting what would follow from a candidate action. Latent world models make these predictions in a representation of the observation, allowing a planner or learned policy to use predicted states without operating directly on future images (Ha and Schmidhuber, 2018; Hafner et al., 2019; 2020). For this approach to support control, the predictions must preserve the information needed to choose useful actions.

Joint-Embedding Predictive Architectures (JEPAs) provide a way to learn by predicting representations without reconstructing the observation itself (LeCun, 2022; Assran et al., 2023; Bardes et al., 2024). The progression from image and video representation learning to the action-conditioned V-JEPA 2-AC model connects this idea to robotic planning (Assran et al., 2025). Variational JEPA (VJEPA) develops a probabilistic formulation in which future representations have a predictive distribution (Huang, 2026). These advances address how to represent and predict the future. They also motivate a further question: which prediction errors should receive the greatest penalty when the model will be used for action?

A latent prediction loss measures discrepancy between predicted and observed representations. Its geometric scale need not match the importance of an error to a controller. For a gripper approaching a block, an error in their relative position may change the appropriate action, while an appearance change in the background may leave that action unchanged (Figure 1). Predicting in latent space does not by itself ensure that the loss distinguishes these cases. Modeling uncertainty also leaves this issue open: a distribution over future states still needs to be evaluated according to the information that matters for action. We call this mismatch between latent prediction error and policy behavior the *prediction–action gap*.

We investigate a policy as a source of this missing training signal. Decision-equivalent latent prediction retains a latent regression loss and adds a Kullback–Leibler (KL) penalty between a frozen reference policy's action

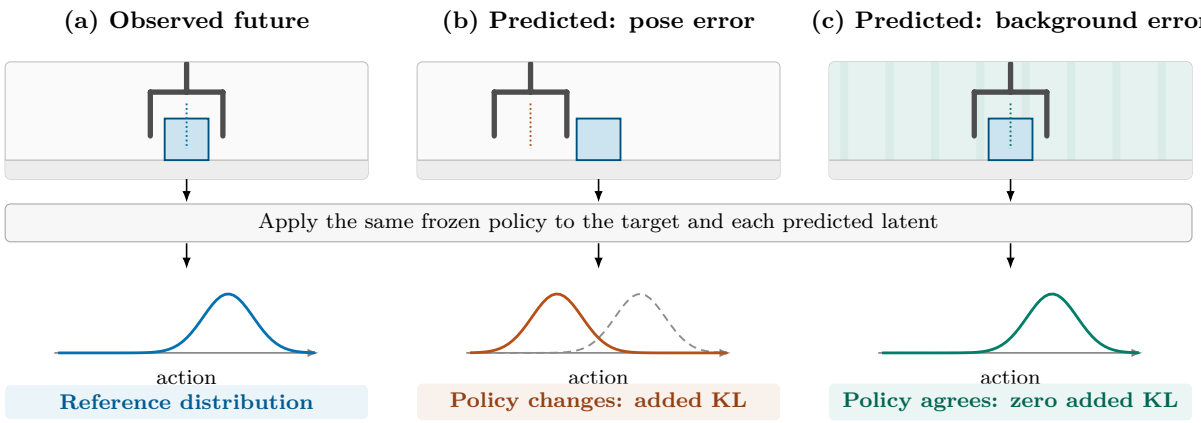


Figure 1: **Prediction errors matter through their effect on a fixed policy.** (a) The observed future provides the reference policy output. For the illustrative controller, (b) a pose error changes the action distribution, while (c) a background-only error leaves it unchanged. The added policy KL distinguishes these cases; the latent prediction term remains active for both predictions. Dashed curves repeat the reference density. Scene sketches stand in for latent states, not decoded predictions. All curves are conceptual, not measured outputs, and imply no latent-MSE ranking.

distributions at predicted and observed future representations. An error receives an additional penalty when it changes what this policy would do. The reference policy is trained by behavior cloning from demonstrations and then held fixed during predictor training. The criterion draws on behavioral state equivalence (Ferns et al., 2004), but measures agreement with one policy rather than equivalence under the full reward and transition structure.

The released implementation uses deterministic predictions with the auxiliary input fixed to zero. A separate posterior branch receives a KL regularizer, but does not supply samples to the prediction loss. We therefore study policy supervision for this point predictor, rather than claim a test of the full variational objective. For traceability, tables and plots retain the release labels *VJEPA* for the latent-only baseline and *DE-VJEPA* for its policy-KL variant.

The evaluation separates preservation of policy behavior from improvement in control. MetaWorld is a simulated robot-arm benchmark covering tasks such as reaching, drawer opening, and pick-and-place (Yu et al., 2020). PushT is a planar manipulation task in which a pusher must move a T-shaped block into a target position and orientation (Chi et al., 2023).

We compare predictors on demonstrations from five MetaWorld tasks and the LeRobot PushT dataset, using evaluation policies trained with seeds different from the reference policy's seed. On clean MetaWorld, DE-VJEPA has 15.4% higher latent mean squared error (MSE), yet 19.8% lower evaluation-policy KL than VJEPA: the two metrics favor different models. Reference-policy and action-mean controls test the source of this improvement, while simple predictors provide comparisons beyond the trained VJEPA baseline.

We then test whether a planner can use the predictions to improve task success. Across 14,850 closed-loop episodes, the offline gains do not establish a reliable aggregate success improvement over VJEPA. The evidence therefore supports a specific contribution: a policy can supply an action-relevant prediction criterion, but improvement under that criterion requires a separate test of the complete control system. The experiments use a frozen ResNet-18 encoder to study the predictor objective; they do not evaluate adaptation of Meta's V-JEPA video backbone.

2 Background

2.1 Latent World Models

A world model predicts the consequences of actions. In a control system, these predictions can be used to compare candidate actions before executing one in the environment. PlaNet demonstrates planning through learned latent dynamics, while Dreamer uses imagined latent trajectories to learn behavior (Hafner et al., 2019; 2020). The model supplies predictions; a planning or policy-learning objective determines how to use them.

A *latent representation* is a vector computed from an observation. Predicting this vector allows the model to operate in a learned feature space. Let o_t denote the observation at time t, and let the encoder f_ϕ produce $z_t = f_\phi(o_t)$. Given this representation and a sequence of actions, a predictor g_θ estimates the representation k steps ahead:

$$\hat{z}_{t+k} = g_\theta(z_t,\, a_{t:t+k}).$$

Here, $a_{t:t+k}$ denotes the intervening actions, and the hat marks a prediction. The target $z_{t+k} = f_\phi(o_{t+k})$ is computed from the future observation in a recorded trajectory. A prediction objective $\mathcal{L}_{\text{pred}} = d(\hat{z}_{t+k}, z_{t+k})$, such as squared Euclidean or cosine distance, trains the predictor to approach this target. Repeated application can generate a predicted trajectory for a candidate action sequence. Its usefulness for control depends on both the information in the representation and the errors the predictor makes.

2.2 Deterministic JEPA

JEPA predicts a target representation from a context representation, without requiring a pixel decoder (LeCun, 2022; Assran et al., 2023). In the temporal, action-conditioned setting used here, the context is the current latent and a sequence of actions; the target is the encoded future state.

The deterministic predictor returns one vector $\hat{z}_{t+k}$ and is trained to match the target z_{t+k}, as in Section 2.1. This point prediction gives one estimated future without explicitly describing the range of possible futures or their relative probabilities.

For example, a single image may not reveal an object's velocity, so its next position can remain uncertain even when the action is known. This motivates replacing a single predicted representation with a distribution over future representations, which is the role of Variational JEPA below.

2.3 Variational JEPA and Implementation Scope

Variational JEPA uses an auxiliary variable h to represent uncertainty over future latents and trains a target-conditioned posterior with a variational objective (Huang, 2026). Appendix H.1 gives that formulation. Our released implementation instead fixes $h = 0$ for prediction and penalizes the posterior separately. Its archived VJEPA and DE-VJEPA labels therefore identify the point predictor and its policy-KL variant; they do not establish optimization of the full variational bound or learned stochastic uncertainty. This setting isolates the effect of policy supervision on a deterministic prediction path.

2.4 Behavioral Similarity and Bisimulation

A behavioral criterion compares states by the consequences of actions. Bisimulation groups states that are interchangeable for a task (Ferns et al., 2004). For every action, two states in the same group must give the same immediate reward and the same probabilities of reaching other such groups.

For example, two manipulation scenes may differ in wall color yet remain interchangeable if object motion and rewards are unchanged under all actions. Moving the object could break that equivalence: the same push could then have a different outcome. The distinction concerns task behavior, not visual similarity.

Bisimulation metrics turn this idea into a graded measure of behavioral difference. Deep bisimulation methods use such a measure to train an encoder (Zhang et al., 2021). We use a simpler policy-dependent comparison: DE-VJEPA compares a frozen policy's action distributions and changes the predictor loss while

keeping the encoder fixed. Agreement with one policy alone does not establish bisimulation or equal long-run value.

Positioning. Our policy-dependent criterion complements feature-prediction objectives and goal- or value-based world-model planning (Bardes et al., 2024; Garrido et al., 2024; Destrade et al., 2026). We hold the encoder and reference policy fixed to isolate the predictor's training signal. Appendix H.2 discusses the broader connections to representation learning, imitation, and control.

3 Decision-Equivalent Latent Prediction (Our Method)

The method retains latent prediction and adds a comparison of policy outputs. The reference policy is held fixed: its role is to supply a training signal for the predictor, rather than to adapt its own decisions to the prediction. Figure 2 shows the training and evaluation paths.

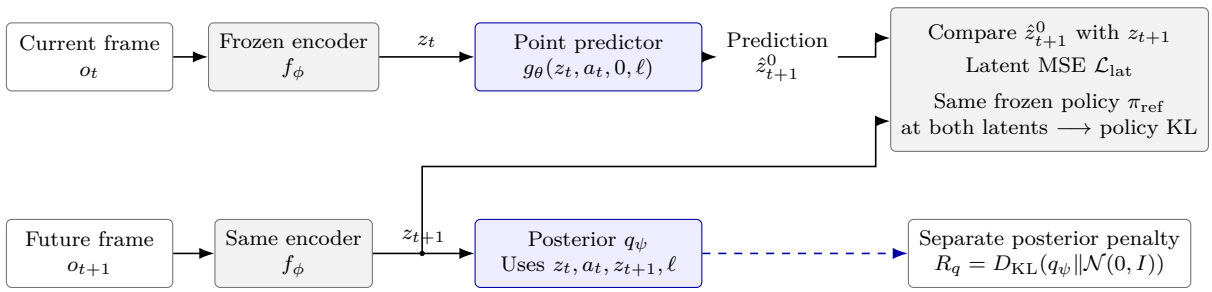
3.1 Decision-Equivalence Loss

Let $\pi_{\text{ref}}(a \mid z, \ell)$ be a frozen reference policy conditioned on latent state z and task context ℓ. In the reported experiments, ℓ is a task-identity embedding rather than a natural-language instruction. We compare the policy's action distribution at the predicted latent $\hat{z}$ with its distribution at the target latent z:

$$d_\pi(\hat{z}, z) = D_{\text{KL}}(\pi_{\text{ref}}(\cdot \mid z, \ell) \,\|\, \pi_{\text{ref}}(\cdot \mid \hat{z}, \ell)) \,. \tag{1}$$

This is an asymmetric divergence, not a metric. Its direction places the observed future latent on the target side: an action assigned high probability there should also receive probability at the predicted latent. We call two latents *decision-equivalent* when their full action distributions agree for the fixed task context.

(a) Predictor training: compare future latents and reference-policy decisions



$$\mathcal{L}_{\text{hybrid}} = \mathcal{L}_{\text{lat}} + \beta R_q + \lambda D_{\text{KL}}\big(\pi_{\text{ref}}(\cdot \mid z_{t+1}, \ell) \,\|\, \pi_{\text{ref}}(\cdot \mid \hat{z}^0_{t+1}, \ell)\big)$$

Blue: updated parameters. Gray: frozen encoder/policy. Prediction uses $h = 0$; dashed path: posterior KL only.

(b) Evaluation: predict without the future target, then score with another policy seed

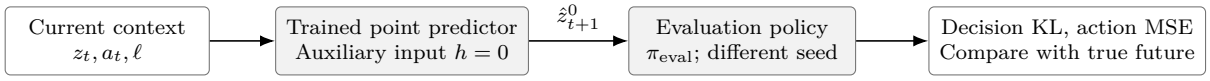


Figure 2: **The same policy compares predicted and observed future latents.** (a) Prediction fixes the auxiliary input to $h = 0$. The latent loss and the same frozen reference policy compare predicted and observed future latents; policy-loss gradients reach the predictor. The target-conditioned posterior has a separate KL penalty and no sample path into these prediction losses. (b) Evaluation uses the same deterministic prediction without future-target inputs and a policy trained with another seed. The experiments use a ResNet-18 encoder and a task ID for context ℓ.

3.2 Evaluated Hybrid Objective

Policy agreement alone does not determine the future latent: different latents can produce the same action distribution, and the reference policy can overlook distinctions that matter to the task. We therefore retain the latent regression loss to anchor training to the observed future and add policy KL to emphasize errors that affect the reference policy's behavior.

In the following implementation equations, z_t and z_{t+1} denote latents standardized using the source training split; input actions are also standardized before prediction (Appendix B). The released one-step implementation fixes the auxiliary input to zero in both training and evaluation:

$$\hat{z}^0_{t+1} = g_\theta(z_t, a_t, 0, \ell), \qquad \mathcal{L}_{\text{lat}} = \frac{1}{D}\|\hat{z}^0_{t+1} - z_{t+1}\|_2^2,$$

where D is the latent dimension. The superscript 0 identifies the fixed input; it is not a prediction obtained by averaging stochastic rollouts. The code also fits a target-conditioned Gaussian posterior $q_\psi(h \mid z_t, a_t, z_{t+1}, \ell)$ and penalizes its departure from the standard-normal prior. Write this penalty as $R_q = D_{\text{KL}}(q_\psi \| \mathcal{N}(0, I))$. The implemented objective is the minibatch average of

$$\mathcal{L}_{\text{hybrid}} = \mathcal{L}_{\text{lat}} + \beta R_q + \lambda\, D_{\text{KL}}\big(\pi_{\text{ref}}(\cdot \mid z_{t+1}) \,\big\|\, \pi_{\text{ref}}(\cdot \mid \hat{z}^0_{t+1})\big)\,. \tag{2}$$

The latent term keeps predictions close to their encoded targets. The policy term gives additional weight to errors that change the reference policy's action distribution, with λ controlling that weight. Setting $\lambda = 0$ gives the released latent-only baseline. Task context ℓ is suppressed in Eq. (2) for readability.

The encoder and reference policy remain frozen. The policy is still differentiated with respect to its predicted-latent input, so the decision loss can update the predictor without changing the policy's parameters.

The prediction and policy losses do not backpropagate through the posterior network; that network receives gradients only from R_q. The task embedding is shared and may receive gradients from both branches. The future latent is used as a loss target and as input to the posterior, but is not an input to the deterministic prediction path during training or evaluation.

This objective does not optimize the posterior expectation in Eq. (14) and does not establish learned stochastic uncertainty. The release configurations associate the core DE-VJEPA results with this hybrid loss; Appendix B.3 states the limits of the historical run provenance. Unevaluated objective variants are collected in Appendix F.

Interpretation. The decision term is zero when the reference policy assigns the same action distribution to the predicted and target latents. The hybrid loss still penalizes latent error within such an equivalence class. Conversely, a reference policy that misses a task-relevant distinction supplies no added signal for preserving it. Appendix G gives the formal definition, proof, and limitations of this policy equivalence.

4 Experiments

We organize the evidence into three comparisons:

1. **Matched prediction study:** compare methods on MetaWorld and PushT using latent error and policy error.
2. **Pilot ablations:** examine the loss weight, reference policy, and action-mean alternative within a separate pilot comparison.
3. **Control tests:** measure task success when planners use the predictions, with multi-step diagnostics to aid interpretation.

The matched study and pilot have their own latent-only reference results. We identify the study for each comparison below.

4.1 Setup

Figure 3 separates the training procedure from the two evaluation protocols. The offline tests measure prediction errors on recorded trajectories. The closed-loop tests instead execute actions in the simulator and measure whether tasks are completed.

We use 33,016 frames from 500 successful demonstrations across five MetaWorld tasks (Yu et al., 2020): REACH, BUTTON-PRESS, DRAWER-OPEN, DOOR-OPEN, and PICK-PLACE. We encode images with a frozen ImageNet ResNet-18 and train Gaussian behavior-cloned policies to predict demonstration actions from the encoded state and task identity. Each trained policy then supplies a fixed reference for its corresponding one-step latent predictor. This setup studies the predictor loss without updating the visual encoder.

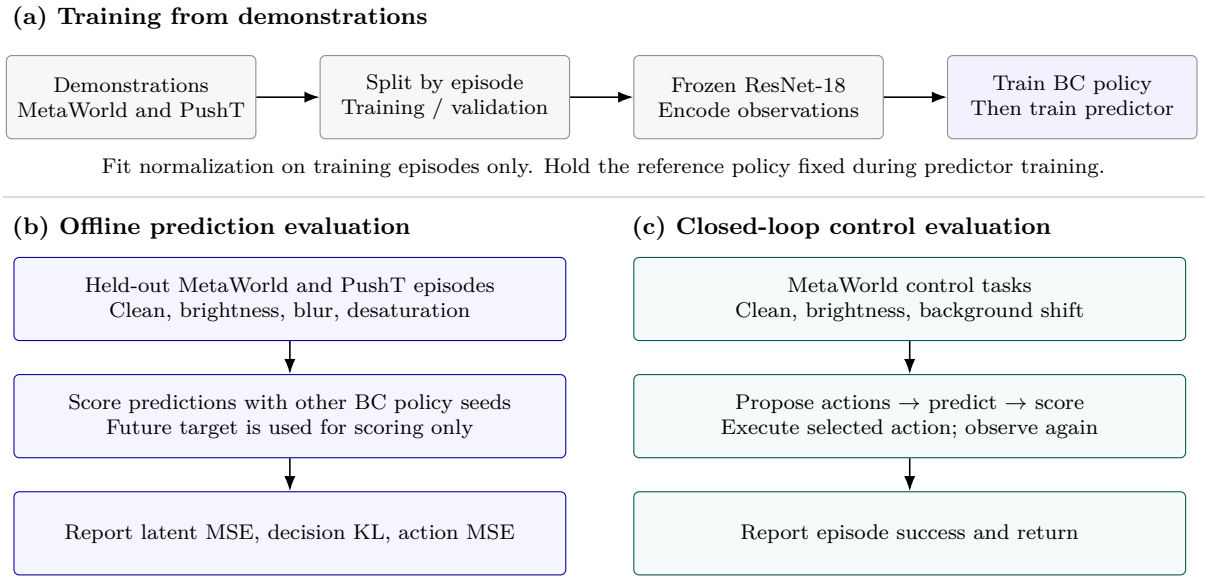


Figure 3: **Prediction fidelity and task completion are evaluated separately.** (a) MetaWorld has 500 demonstrations across five tasks; PushT has 206 episodes of one task. For each seed in $\{42, 1000, 10000\}$, episodes are split 80/20 for training and validation. (b) Each predictor is scored by the two BC policies with other seeds. These evaluations share trained models and are not independent training replicates. (c) Closed-loop tests use REACH, BUTTON-PRESS, and DRAWER-OPEN, with a background shift alongside clean and brightness conditions.

Model names. We retain the release's model labels in every table and plot. VJEPA is its latent-only point predictor with $h = 0$; DE-VJEPA adds the frozen-policy KL. These labels do not imply optimization of the variational bound in Eq. (14). BJEPA combines a dynamics Gaussian at $h = 0$ with a task prior, and DE-BJEPA adds policy KL to that combination. Its Gaussian NLL is distinct from posterior-sampled variational training. The encoder is the same frozen ResNet-18 throughout these comparisons.

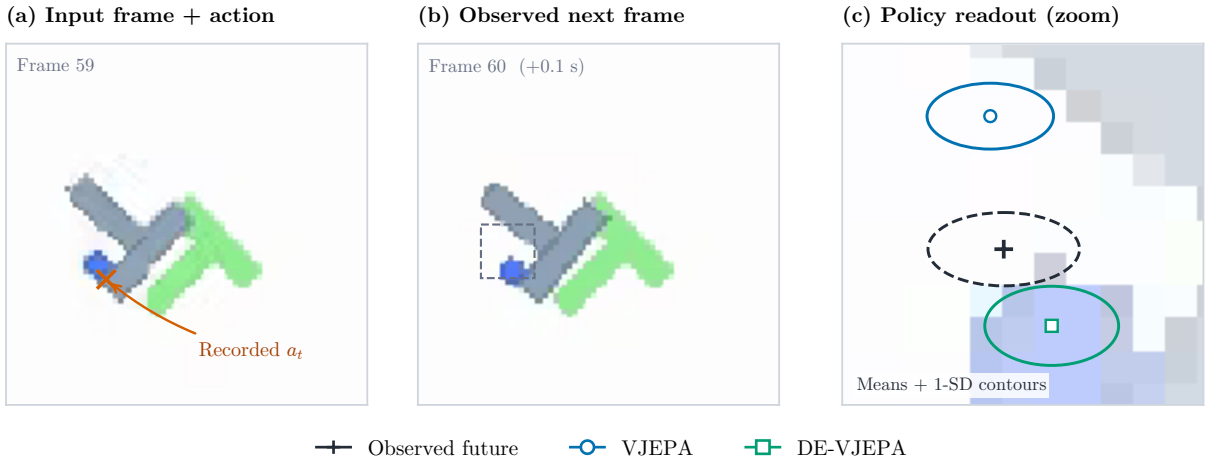
Evaluation policies and metrics. The predictor receives no true future latent during evaluation. To test whether a gain transfers beyond the exact policy used in its loss, we score each predictor with the two BC policies trained using other seeds. Three predictor seeds and two evaluator choices give six source/evaluator pairs; *source* denotes the predictor/reference-policy training seed. Throughout the paper, an *independent evaluator* means one trained with a different seed. It shares the policy architecture and demonstration source; its training data are not guaranteed to be disjoint from those of the reference policy.

We measure latent mean squared error (MSE), forward KL between the evaluation policy's distributions at the true and predicted future latents (*decision KL*), and squared error between the corresponding action means (*action MSE*). Lower values are better for all three metrics. Evaluation uses clean images and

deterministic changes to brightness, blur, and saturation. Appendix B gives the data splits, transformations, and training configuration.

Statistical units. The analysis first averages frame-level errors within each task and source/evaluator pair. On five-task MetaWorld, five tasks and six pairs give 30 task/source/evaluator units per visual condition. These units share predictors and policies; they still represent only three predictor-training seeds. The paired tests compare methods on these units, and their reported p-values are nominal because they do not fully account for this dependence.

A concrete PushT transition. Figure 4 shows what an offline comparison measures. The current image and recorded action are the predictor's inputs; the next image supplies the target latent for scoring. We then apply the same evaluation policy to the observed and predicted future latents. Its action means show where the pusher would be directed next, while its Gaussian contours show the corresponding action uncertainty. These are policy readouts, not decoded predictions of future images. This single transition illustrates the metrics; the aggregate comparisons follow in Section 4.2.



This transition only	VJEPA	DE-VJEPA
Decision KL (lower is better)	8.223	2.153
Action-mean MSE (lower is better)	0.0037	0.0017

Figure 4: **From a recorded transition to a policy-level comparison.** (a) Input frame and recorded target-position action a_t. (b) Observed frame 0.1 s later; the dashed box marks the zoom in (c). (c) The same evaluator policy maps the observed, VJEPA-predicted, and DE-VJEPA-predicted future latents to means and one-standard-deviation Gaussian contours for the *following* action. Real frames and model outputs come from an additional illustrative rerun: source seed 42, evaluator seed 1000, episode 7, frames 59–60. The episode is held out from both training splits; the transition was selected by recorded object motion before training. These values are separate from the aggregate results.

4.2 Decision Preservation Across Two Datasets

The matched study compares 11 predictors on five-task MetaWorld and LeRobot PushT. Alongside these four released models, the controls include copying the current latent, linear dynamics, alternative latent objectives, and policy-based losses. Appendix B.9 defines the complete baseline set. PushT contains 25,650 frames from 206 episodes at 10 Hz. Its two-dimensional actions are absolute pixel targets, which we map from the $[0, 512]$ workspace to $[-1, 1]$ before BC training.

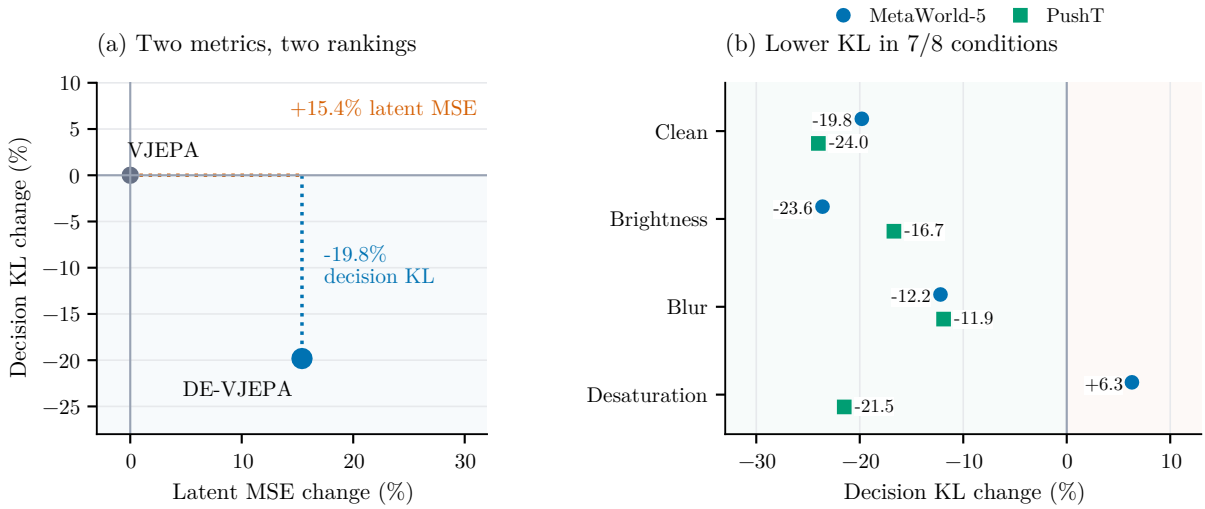


Figure 5: **Latent error and decision error can favor different models.** (a) On clean MetaWorld, DE-VJEPA trades higher latent MSE for lower decision KL relative to VJEPA; changes use Table 9. (b) DE-VJEPA reduces decision KL in seven of eight dataset–condition comparisons. Points and labels use the reported changes in Table 1. Negative changes are better. Multi-step results are shown separately in Figure 12.

Table 1: DE-VJEPA change relative to VJEPA in the matched cross-dataset study. Negative values are improvements. Bold indicates decreases with nominal paired t-test $p < 0.05$; the shared-model dependence is described in the setup.

Dataset	Shift	Latent MSE	Decision KL	Action MSE
MetaWorld-5	Clean	+15.4%	**−19.8%**	**−9.0%**
	Brightness	−0.2%	**−23.6%**	**−6.4%**
	Blur	+1.1%	**−12.2%**	−0.9%
	Desaturation	+1.4%	+6.3%	+0.8%
PushT	Clean	**−3.5%**	**−24.0%**	**−13.7%**
	Brightness	+1.1%	**−16.7%**	**−8.4%**
	Blur	**−1.6%**	**−11.9%**	**−5.1%**
	Desaturation	+0.6%	**−21.5%**	**−12.0%**

The clean MetaWorld result reverses the model ranking: VJEPA has lower latent MSE, while DE-VJEPA has lower decision KL. The 15.4% increase in latent MSE accompanies a 19.8% reduction in decision KL (Table 1). Across both datasets, DE-VJEPA lowers decision KL in seven of eight conditions and improves PushT action MSE in all four. MetaWorld desaturation is the exception. The reported paired tests appear in Appendix B.7.

The simple baselines help put these gains in context. Persistence obtains the lowest clean PushT decision KL (1.65 versus 2.29 for DE-VJEPA). Copying the current latent is therefore a strong baseline on this 10 Hz sequence. Linear dynamics is competitive on several shifted metrics. Cosine prediction substantially worsens latent MSE, although it has the best PushT desaturation decision KL. BJEPA and DE-BJEPA generally improve shifted latent MSE, but their decision gains are less consistent than DE-VJEPA. These controls bound the result: the policy loss often helps relative to trained VJEPA, while simple alternatives still win individual comparisons.

Product-of-Experts extension. BJEPA combines learned dynamics with a task-conditioned Gaussian prior fitted only on training latents. DE-BJEPA adds the same decision loss to that predictor. Their gains depend on the visual condition: DE-BJEPA has the best brightness values in the four-method comparison, while BJEPA has the lowest blur decision KL. Neither improves desaturation decision KL over VJEPA, and neither uniformly dominates DE-VJEPA. Table 9 reports the full values; the paired comparisons are in Appendix B.7.

4.3 Pilot Ablations of the Training Loss

The pilot comparisons use their own latent-only reference results. In the five-task pilot, DE-VJEPA reduces clean decision KL by 21.1%, compared with 19.8% in the matched study. The pilot estimates are used for the loss-design ablations below; they are not pooled with the matched estimates. Table 3 gives the complete pilot reference comparison.

Loss weight. The three-task loss-weight sweep shows a limited useful range. At $\lambda = 10^{-3}$, clean decision KL falls by 12.6% with a 0.9% latent-MSE increase. At $\lambda = 10^{-1}$, latent MSE rises by 97.8% and decision metrics also deteriorate. Appendix A.2 gives the full sweep, including the different behavior under brightness.

Reference policy. A random policy supplies an extra loss without learned task behavior. Comparing frozen random, weak BC, and strong BC references tests whether the gain tracks the quality of the policy defining that loss.

The strong BC reference supplies the largest reduction in decision KL (Figure 6); random and weak references change the baseline much less. This pattern links the signal to reference-policy quality, without ruling out other regularization effects. Exact values are in Table 4.

Action distribution versus action mean. Matching only the policy mean tests a simpler alternative to comparing the full Gaussian action distribution. At selected weights of 0.1 for action matching and $\lambda = 0.01$ for policy KL, clean decision KL falls by 3.3% and 21.1%, respectively; its brightness gains are also smaller. The penalties have different scales, so these results compare the tested settings rather than establish an equal-strength or universal preference. Table 5 and Appendix A.3 give the weights and full comparison.

4.4 Closed-Loop Evaluation and Diagnostics

The control tests measure whether the predictions improve executed actions. The first comparison uses a shared one-step planner on REACH, BUTTON-PRESS, and DRAWER-OPEN, with clean, brightness, and background-shift conditions. A separately trained seed-7 BC policy supplies the controller for all predictor seeds. The comparison contains 1,620 episodes; Table 11 gives the success rates.

Direct BC executes its policy mean. The planner instead proposes nearby actions, predicts their future latents, and selects an action by predicted policy entropy with a penalty for departing from the BC action. It repeats this process after each environment transition. The full settings and algorithm appear in Appendix C.

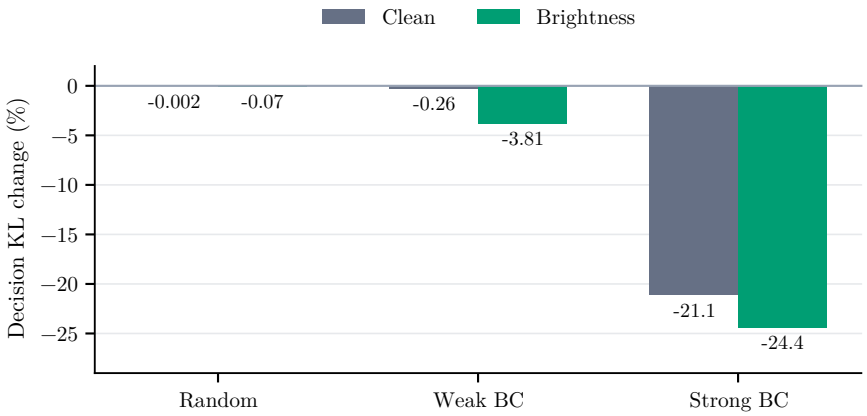


Figure 6: **The strongest reference policy supplies the largest gain.** Five-task pilot changes relative to latent-only prediction, with each reference held fixed during predictor training. Lower is better. Table 4 gives the values.

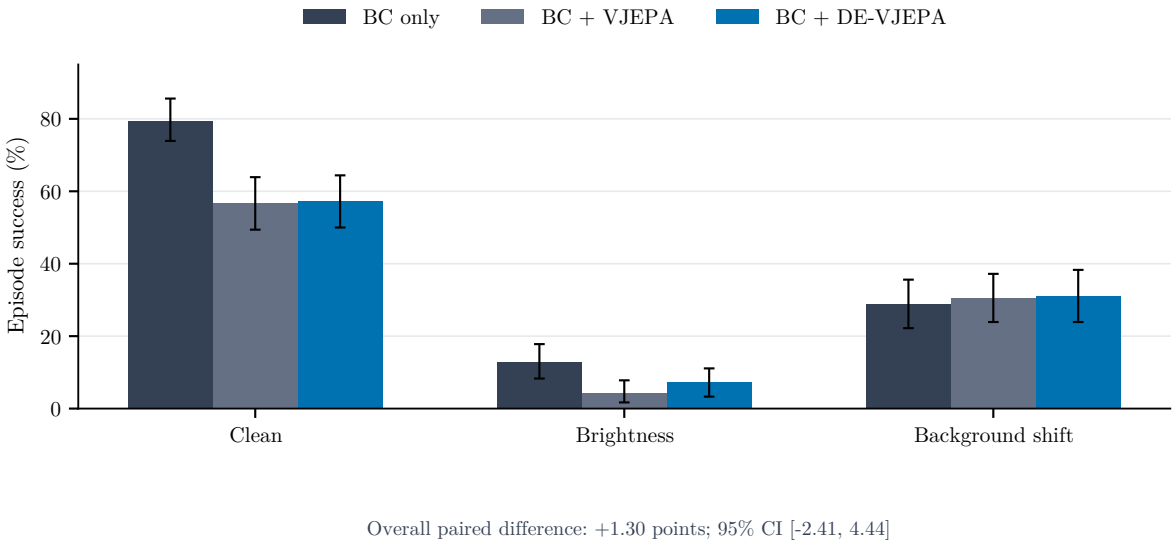


Figure 7: **The offline advantage does not establish a control gain.** Bars and error bars show the reported success means and 95% episode-bootstrap intervals from Table 11, with 180 episodes per method and condition. Direct BC is strongest on clean observations; the aggregate paired DE-VJEPA–VJEPA difference is not significant. Task-level results appear in Table 12.

The aggregate DE-VJEPA–VJEPA success difference is +1.30 percentage points ($p = 0.473$; bootstrap 95% CI $[-2.41, 4.44]$). This does not establish a control gain. Direct BC remains the strongest clean controller, exceeding BC+DE-VJEPA by 22.22 success points. Appendix C reports return differences and condition-level tests.

A subsequent 4,860-episode ablation tests whether changing the planner score helps. Four candidate-selection rules—entropy, decision-distribution consistency, action-mean consistency, and random selection—were evaluated with paired candidate perturbations. No rule yields a significant DE-VJEPA success advantage over VJEPA. Action-mean consistency exactly recovers BC-only success because it selects actions with negligible deviation from the BC mean, whereas entropy and decision-consistency scoring reduce success as their action deviations grow. Appendix D gives the score-ablation results.

Multi-step and progress diagnostics. Table 2 summarizes all four closed-loop studies. The episode counts describe evaluation coverage, not independent training runs. The final study uses the cross-entropy method (CEM) to search action sequences within model-predictive control (MPC). In a separate offline diagnostic, autoregressive prediction on held-out demonstrations retains a DE-VJEPA decision-KL advantage at horizons one, three, and five (20.5%, 12.8%, and 13.7%, respectively), while latent MSE remains higher. This diagnostic uses recorded action sequences rather than actions selected by a planner (Appendix D).

Table 2: Closed-loop study inventory. No study establishes a significant aggregate DE-VJEPA success advantage over matched VJEPA.

Study	Episodes	Principal observation
Local entropy planner	1,620	DE-VJEPA–VJEPA: +1.30 success points, $p = 0.473$.
Local-score ablation	4,860	No tested score establishes a DE-VJEPA advantage; action-mean consistency reproduces BC.
One-step progress planner	4,860	At $\alpha = 0.1$: +0.37 points, $p = 0.161$.
Progress-aware CEM/MPC	3,510	At $H = 3$, $\alpha = 0.1$: +5.56 points, $p = 0.113$. Longer horizons degrade clean success.
Total	14,850	Three tasks; one independently seeded BC controller.

The progress heads predict normalized time within successful demonstrations, not reward-to-go. In the CEM study, horizon three gives the largest mean DE-VJEPA–VJEPA difference, but its nominal paired test is inconclusive. Together, these diagnostics show a gap between preserving policy distributions and selecting actions that complete the task. The learned progress score does not resolve that gap in the present experiments. Appendix E gives configurations, condition-level results, and uncertainty estimates.

5 Limitations

Implementation scope. The released VJEPA-labelled predictor fixes $h = 0$ for its prediction loss; its posterior branch is trained only by KL regularization. The results therefore concern this deterministic implementation and its policy-KL variant, not a full test of Variational JEPA or of learned stochastic uncertainty. The source and aggregate CSVs occur in the same release, but the original run logs and checkpoints are unavailable. We cannot verify the exact code executed for those historical runs (Appendix B.3).

Reference policy quality. The decision term can only emphasize distinctions recognized by π_{ref}. If the policy overlooks a task-critical feature, the penalty provides no signal to preserve it. Evaluation under other seeds tests transfer within the same behavior-cloning architecture and demonstration source, not transfer to arbitrary controllers.

KL direction. We use the forward KL, which penalizes missing probability mass on actions likely under the target policy. Reverse KL has a different, often mode-seeking bias. We have not established that the forward direction is preferable in other policy families or tasks.

Continuous action spaces. Computing d_π exactly requires evaluating KL divergences between continuous distributions, which is simple for Gaussian policy heads but more difficult for flow- or diffusion-based policies (Chi et al., 2023).

Short-horizon limitation. Agreement between one-step action distributions does not ensure agreement about long-run value. Appendix F gives possible multi-step and value-based objectives. Neither is validated by rolling out the one-step-trained models used here.

Statistical scope. Several evaluation units reuse the same trained predictor or policy and draw from the same demonstration source. They are not independent training replications. The reported standard errors and paired p-values use the original analysis units and do not fully account for this dependence. The tests also do not control for all comparisons across metrics, visual conditions, and planner settings. More independent training seeds and an analysis that accounts for the shared models are needed for stronger statistical conclusions.

Empirical scope. The encoder is a frozen ImageNet ResNet-18. The experiments therefore do not establish results for Meta's V-JEPA backbones or other learned representations. The offline demonstrations are all successful, which prevents estimating a relationship between prediction error and demonstration success. MetaWorld desaturation is a negative condition for DE-VJEPA, and persistence is stronger on clean PushT.

The closed-loop studies cover three tasks and one BC controller trained with a separate seed. The progress heads predict normalized demonstration time, which may not track task progress outside the demonstrations. The background shift is a deterministic image transformation, not a change to simulator textures. The tested planners do not establish an aggregate DE-VJEPA success advantage, and longer CEM horizons often reduce clean success. Their results do not support a general claim of improved control or visual robustness.

6 Discussion

Implications for predictor evaluation. The ranking reversal shows that latent MSE alone can miss a change in policy fidelity, even with the encoder fixed. Policy error supplies a complementary criterion whose meaning depends on the evaluator. The reference-policy ablation links the training signal to learned task behavior; the persistence result shows why comparisons should also include simple predictors.

Interpreting the control gap. Entropy and distribution-consistency scores can reward confidence or conservatism without rewarding task progress. Action-mean consistency illustrates this limitation: it matches BC success while barely changing BC's actions. Policy consistency alone therefore need not improve control.

The decision-KL advantage remains through five-step prediction under recorded actions. This rules out its immediate disappearance offline, but planners can select other actions and visit other states, where model error remains possible. A progress head trained on normalized demonstration time may also mis-score those states. Reward- or value-based scores, uncertainty estimates, and matched training/planning horizons are distinct follow-up tests. The current results do not isolate one cause of planner failure.

7 Conclusion

The policy-KL variant, labelled DE-VJEPA in the release, adds a frozen-policy KL term to deterministic latent prediction. It reduces evaluation-policy KL relative to VJEPA in seven of eight MetaWorld/PushT conditions, even though latent MSE can increase. The controls show why reference-policy quality matters, while closed-loop tests leave the control benefit unresolved. These findings support evaluating latent fidelity, policy fidelity, and task completion as separate properties of a world-model control system.

References

M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023.

M. Assran, A. Bardes, Q. Garrido, and others. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv:2506.09985*, 2025.

A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv:2404.08471*, 2024.

A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, and others. RT-1: Robotics transformer for real-world control at scale. *arXiv:2212.06817*, 2022.

J. Bruce, M. Dennis, A. Edwards, and others. Genie: Generative interactive environments. *arXiv:2402.15391*, 2024.

C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.

P. Dan, K. Kedia, A. Chao, E. W. Duan, M. A. Pace, W.-C. Ma, and S. Choudhury. X-Sim: Cross-embodiment learning via real-to-sim-to-real. *arXiv:2505.07096*, 2025.

M. Destrade, O. Bounou, Q. Le Lidec, J. Ponce, and Y. LeCun. Value-guided action planning with JEPA world models. *arXiv:2601.00844*, 2026.

N. Ferns, P. Panangaden, and D. Precup. Metrics for finite Markov decision processes. In *UAI*, pages 162–169, 2004.

N. Ferns, P. Panangaden, and D. Precup. Bisimulation metrics for continuous Markov decision processes. *SIAM Journal on Computing*, 40(6):1662–1714, 2011.

Q. Garrido, M. Assran, N. Ballas, A. Bardes, L. Najman, and Y. LeCun. Learning and leveraging world models in visual representation learning. *arXiv:2403.00504*, 2024.

C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.

D. Ha and J. Schmidhuber. World models. *arXiv:1803.10122*, 2018.

D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *ICML*, 2019.

D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. In *ICLR*, 2020.

D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering Atari with discrete world models. In *ICLR*, 2021.

D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv:2301.04104*, 2023.

N. Hansen, X. Wang, and H. Su. Temporal difference learning for model predictive control. In *ICML*, 2022.

N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. In *ICLR*, 2024.

K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2022.

B. Hou, G. Li, J. Jia, T. An, X. Guo, S. Leng, H. Geng, Y. Ze, T. Harada, P. Torr, O. Mees, M. Pollefeys, Z. Liu, J. Wu, P. Abbeel, J. Malik, Y. Du, and J. Yang. World model for robot learning: A comprehensive survey. *arXiv:2605.00080*, 2026.

A. Hu, L. Russell, H. Yeo, Z. Murez, G. Fedoseev, A. Kendall, J. Shotton, and G. Corrado. GAIA-1: A generative world model for autonomous driving. *arXiv:2309.17080*, 2023.

Y. Huang. VJEPA: Variational joint embedding predictive architectures as probabilistic world models. *arXiv:2601.14354*, 2026.

K. Kedia, P. Dan, A. Chao, M. A. Pace, and S. Choudhury. One-shot imitation under mismatched execution. *arXiv:2409.06615*, 2024.

Y. LeCun. A path towards autonomous machine intelligence. OpenReview, 2022.

A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv:1807.03748*, 2018.

S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.

J. Schrittwieser, I. Antonoglou, T. Hubert, and others. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

K. Q. Weinberger and L. K. Saul. Distance metric learning for large margin nearest neighbor classification. *Journal of Machine Learning Research*, 10:207–244, 2009.

S. Yang, J. Walker, J. Parker-Holder, and others. Learning interactive real-world simulators. *arXiv:2310.06114*, 2023.

T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *CoRL*, 2020.

A. Zhang, R. McAllister, R. Calandra, Y. Gal, and S. Levine. Learning invariant representations for reinforcement learning without reconstruction. In *ICLR*, 2021.

T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *RSS*, 2023.

B. Zitkovich, T. Yu, S. Xu, and others. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *CoRL*, 2023.

A Additional Offline Comparisons

This appendix provides the additional pilot plots and baseline comparisons discussed in the main text.

A.1 Pilot Reference Comparison

The following tables use the pilot's latent-only reference. They support the loss-design comparisons in the main text and are separate from the matched cross-dataset estimates.

Table 3: Five-task hybrid DE-VJEPA at $\lambda = 0.01$ relative to the released latent-only point predictor. Negative changes are improvements. Bold indicates decreases with nominal paired t-test $p < 0.05$.

Condition	Metric	Change	paired p
Clean	Latent MSE	+15.6%	3.9×10^{-5}
Clean	Decision KL	**−21.1%**	3.1×10^{-6}
Clean	Action MSE	**−8.0%**	0.0097
Brightness	Latent MSE	+0.03%	0.889
Brightness	Decision KL	**−24.4%**	0.0188
Brightness	Action MSE	**−6.0%**	0.0456
Blur	Decision KL	−13.5%	0.0524
Desaturation	Decision KL	+6.7%	0.133

On clean data, latent MSE rises while both evaluation-policy errors fall (Table 3). This is the trade-off described by the prediction–action gap. Under brightness, policy errors fall with little change in latent MSE. The blur result does not meet the nominal 0.05 threshold for decision KL, and desaturation shows no improvement.

A.2 Loss-Weight Sweep

On the three-task subset (REACH, BUTTON-PRESS, and DRAWER-OPEN), we sweep $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 3\times10^{-2}, 10^{-1}\}$. The weight changes the balance between latent fidelity and decision error. On clean data, $\lambda = 10^{-3}$ reduces decision KL by 12.6% and action MSE by 6.0%, while increasing latent MSE by only 0.9%. At $\lambda = 10^{-2}$, clean latent MSE increases by 20.8%; at $\lambda = 10^{-1}$ it increases by 97.8% and decision metrics also deteriorate. Under brightness, larger values ($\lambda = 0.03$–0.1) reduce decision KL by 12.4–14.6%, but incur a 4.0–10.5% latent-MSE penalty and do not significantly improve action MSE.

A.3 Reference Policy and Action-Mean Controls

Table 4: Decision KL change relative to latent-only prediction at $\lambda = 0.01$.

Reference geometry	Clean	Brightness
Random frozen policy	−0.002%	−0.07%
Weak BC policy	−0.26%	−3.81%
Strong BC policy	**−21.1%**	**−24.4%**

Action-mean matching. We compare policy KL with the simpler regularizer $\|\mu_{\text{ref}}(z) - \mu_{\text{ref}}(\hat{z})\|^2$, sweeping its weight from 10^{-4} to 10^{-1}. On clean data, action-mean matching at weight 0.1 reduces action MSE by 8.8%, but reduces decision KL by only 3.3%. At equal weight 0.01, policy KL reduces clean decision KL by 21.1%, compared with 0.45% for action matching. Under brightness, all action-matching weights change decision KL and action MSE by less than 0.05%, whereas policy KL improves them by 24.4% and 6.0%, respectively.

Table 5: Percentage changes from latent-only VJEPA for selected pilot settings; negative is better. Columns report clean latent MSE, clean decision KL, and brightness decision KL. Action-mean matching uses weight 0.1 and DE-VJEPA uses $\lambda = 0.01$.

Method	**Lat.**	**Dec.**	**Bright. dec.**
VJEPA	base	base	base
Action mean	≈ 0	-3.3	≈ 0
DE-VJEPA	$+15.6$	$\mathbf{-21.1}$	$\mathbf{-24.4}$

Interpretation. Policy KL gives the larger reduction in decision KL in these pilot settings, especially under brightness. The two penalties have different scales, so an equal numerical weight is not an equal-strength comparison. The sweeps compare their observed behavior over the tested ranges; they do not establish that policy KL is preferable for every choice of weights or metrics.

A.4 Additional Plots and Core Comparison

Table 6: VJEPA/BJEPA comparison. Values are mean ± standard error across 30 task/source/evaluator units. Lower is better.

Shift	**Method**	**Latent MSE**	**Decision KL**	**Action MSE**
Clean	VJEPA	$.00151 \pm .00007$	$.350 \pm .024$	$.000734 \pm .000130$
	BJEPA	$.00205 \pm .00014$	$.426 \pm .023$	$.001000 \pm .000159$
	DE-VJEPA	$.00174 \pm .00007$	$\mathbf{.280 \pm .021}$	$\mathbf{.000668 \pm .000112}$
	DE-BJEPA	$.00205 \pm .00015$	$.361 \pm .021$	$.000932 \pm .000149$
Brightness	VJEPA	$.05121 \pm .00059$	308.5 ± 66.4	$.07941 \pm .00573$
	BJEPA	$.04878 \pm .00074$	215.5 ± 41.9	$.07248 \pm .00534$
	DE-VJEPA	$.05113 \pm .00062$	235.8 ± 52.8	$.07436 \pm .00546$
	DE-BJEPA	$\mathbf{.04731 \pm .00073}$	$\mathbf{206.2 \pm 38.4}$	$\mathbf{.07089 \pm .00512}$

A.5 Comparison with Alternative Methods

Table 7 summarizes the baseline comparisons. The repeated finding is lower decision KL relative to trained VJEPA across most tested conditions. Other methods achieve lower errors in individual conditions, and the table should not be read as a universal ranking.

These comparisons support a specific claim: adding policy KL often reduces decision error relative to the released latent-only baseline. Persistence, cosine prediction, and the Bayesian variants remain useful controls because they perform better in some conditions.

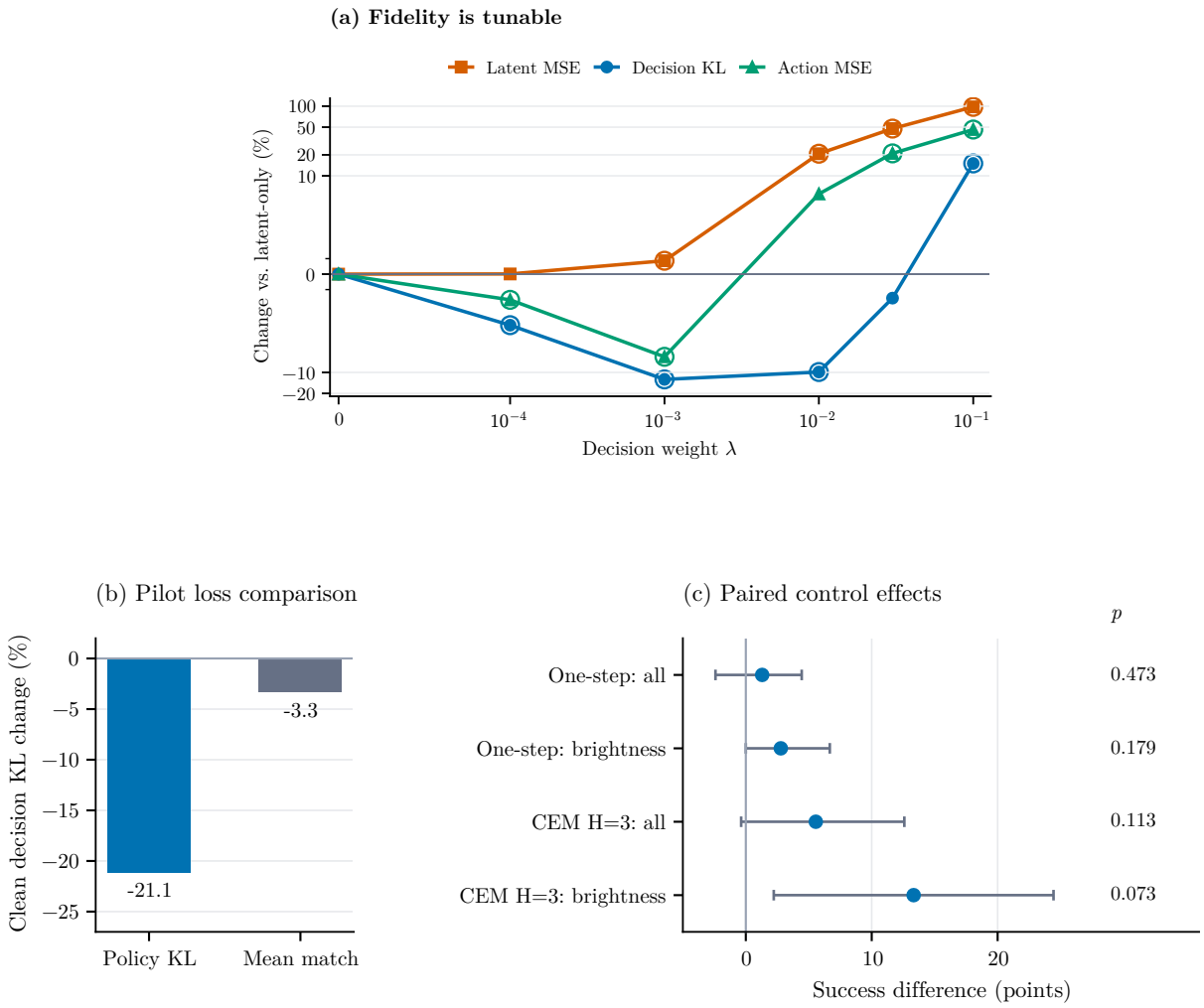


Figure 8: Pilot ablations and control uncertainty. (a) The loss-weight panel uses the original compact results; rings denote nominal paired $p < 0.05$. (b) Clean decision-KL changes from Table 5, using the selected loss weights specified there. (c) Reported paired DE-VJEPA–VJEPA success differences and bootstrap 95% intervals, with nominal paired-test p-values in a separate column. The brightness CEM interval and paired test give different threshold decisions, as discussed in Appendix E.

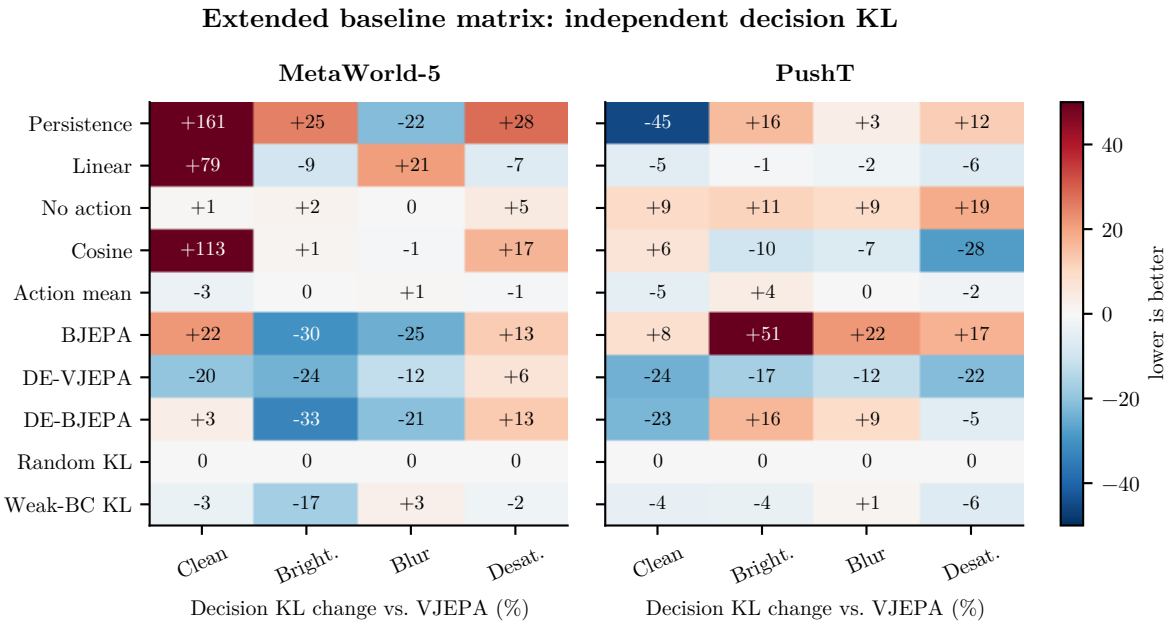


Figure 9: Ten-method decision-KL comparison. Each cell reports percentage change from matched VJEPA; blue is better. Colors are clipped to ±50%, while printed values are not clipped. Persistence and cosine prediction win individual PushT conditions, and Bayesian variants are strongest for selected MetaWorld shifts.

Table 7: Positioning of DE-VJEPA relative to evaluated alternatives. "Better" means lower error on the relevant metric.

Comparator	**Observed comparison with DE-VJEPA**
VJEPA	DE-VJEPA lowers independent decision KL in seven of eight dataset–condition comparisons, including all four PushT conditions, although latent MSE can increase.
Persistence	Persistence is stronger on clean PushT decision KL (1.65 versus 2.29), but is substantially less reliable under visual shifts and on multi-task MetaWorld.
Linear dynamics	The linear model is competitive on some shifted latent and action metrics, but does not provide the same consistent decision-KL improvement over trained VJEPA.
No-action and cosine JEPA	These controls win isolated cells, including cosine JEPA on PushT desaturation decision KL, but their overall latent or decision performance is less stable.
Action-mean matching	Matching policy means helps in some settings. Policy KL has stronger brightness results in the pilot and compares the full Gaussian action distribution, including its variance.
Random / weak policy KL	Random policy geometry provides no reliable benefit, and weak-BC geometry is generally less effective than the strong-BC reference.
BJEPA	The Product-of-Experts predictor lowers shifted latent error in some conditions, while its decision-KL gains vary.
DE-BJEPA	Combining Bayesian prediction with decision regularization is competitive and can reduce the latent-fidelity penalty, but does not uniformly dominate DE-VJEPA in the current experiments.

B Reproducibility Appendix

This appendix records the reported experimental protocol and output filenames. The source code and compact result summaries were recovered from the original project release. Filenames for large raw outputs and checkpoints identify artifacts excluded from that release; those files are not included here.

B.1 Dataset and Evaluation Units

The archive contains 33,016 encoded frames from 500 successful MetaWorld demonstrations, with 100 episodes for each of REACH, BUTTON-PRESS, DRAWER-OPEN, DOOR-OPEN, and PICK-PLACE. Removing transitions that cross episode boundaries gives 32,516 valid one-step prediction examples before splitting. Observations are rendered at 224×224 from the `corner2` camera and encoded once by a frozen ImageNet ResNet-18 into 512-dimensional latents. Actions are four-dimensional continuous MetaWorld controls.

For each seed in $\{42, 1000, 10000\}$, episodes are shuffled and split 80/20 into training and validation sets. Splitting is performed by episode rather than by frame. Each trained predictor is evaluated by the two behavior-cloned evaluator policies whose seeds differ from its training/reference-policy seed. Consequently, each visual condition contains 5 tasks $\times 3$ training seeds $\times 2$ evaluator seeds $= 30$ paired analysis units. The reported tests average frame-level metrics within each task/source/evaluator unit, avoiding the treatment of individual frames as independent replicates. The resulting units still share trained predictors and policies, so this aggregation does not remove all dependence.

PushT. The second dataset is the official LeRobot PushT release (https://huggingface.co/datasets/lerobot/pusht). It contains 25,650 frames from 206 single-task episodes, with 96×96 RGB observations, two-dimensional agent state, and two-dimensional target-position actions at 10 Hz. We decode the provided AV1 video, retain the published episode indices, and encode each frame with the same frozen ResNet-18 used for MetaWorld. PushT actions are pixel coordinates with observed range approximately $[12, 511]$; they are transformed as $a' = a/256 - 1$ so the bounded Gaussian policy mean can represent the action domain. An initial unnormalized run caused policy collapse and is explicitly archived as invalid; none of its metrics is used in the paper.

PushT uses the same three training seeds and two non-matching evaluator seeds per source seed. Because it has one task, each condition has six paired source/evaluator units rather than MetaWorld's 30 task/source/evaluator units.

Illustrative transition. Figure 4 uses a separate clean-PushT rerun of the original implementation, with source seed 42 and evaluator seed 1000. We train both BC policies for 30 epochs and the VJEPA/DE-VJEPA predictors for 50 epochs, using the settings in Table 8 and the unbanded hybrid loss with $\lambda_{\mathrm{DE}} = 0.01$. This rerun uses float32 on an Apple GPU via MPS, with no AMP and no data-loader workers. It does not regenerate the aggregate results. The dataset is pinned to LeRobot PushT revision `a16737ffb6c8dbe935291b9fd2f36d14c9bf0820`. Episode 7 is the first episode held out under both seeds. Before training, we select its first transition with maximal change in the gray object's pixel silhouette. At evaluation, predicted latents are returned to the encoder's scale and then normalized using the evaluator's own training statistics. The source package includes the two RGB frames, saved policy outputs, selection rule, and a script that rebuilds the figure without retraining.

B.2 Visual Conditions

Clean evaluation uses the frozen archive without image modification. The three held-out image transformations are deterministic: brightness is multiplied by 0.55, Gaussian blur uses radius 2.0, and color saturation is multiplied by 0.15. The same frame ordering, episode identifiers, actions, and task labels are retained in every shifted archive, allowing sample-aligned evaluation.

B.3 Model and Optimization Details

Table 8: Concrete implementation and training configuration used in the reported four-method comparison.

Component	Setting	Details
Encoder	Frozen ResNet-18	ImageNet weights; 512-D latent; no fine-tuning
Reference/evaluator policy	Gaussian BC	2-layer MLP, width 512, SiLU, 32-D task embedding
Policy output	Diagonal Gaussian	Tanh mean; log standard deviation clipped to $[-5, 1]$
Policy training	AdamW	30 epochs, batch 1024, lr 3×10^{-4}, weight decay 10^{-5}
Predictor context	z_t, a_t, ℓ	One-step horizon; 32-D task embedding
Auxiliary branch	Gaussian h	64 dimensions; log variance clipped to $[-10, 5]$
Predictor network	Residual MLP	2 hidden layers, width 1024, LayerNorm, SiLU
Predictor training	AdamW	50 epochs, batch 512, lr 3×10^{-4}, weight decay 10^{-5}
Regularization	Gradient clipping	Global norm clipped to 5.0; AMP enabled
Posterior KL weight	$\beta = 10^{-3}$	KL from posterior $q(h \mid z_t, a_t, z_{t+1})$ to standard normal
Decision weight	$\lambda_{\mathrm{DE}} = 10^{-2}$	Used by DE-VJEPA and DE-BJEPA
BJEPA prior	Empirical task Gaussian	Per-task mean/variance from normalized training latents only
BJEPA objective	Gaussian NLL	Diagonal Gaussian NLL plus separate posterior KL
Hardware	RTX 4090	Single 24 GB GPU; eight data-loader workers

Latents and actions are standardized using statistics fitted only on the training partition for each seed. The same normalization is stored with each BC checkpoint and reused by its corresponding predictor. The prediction path fixes $h = 0$ during training, validation, and evaluation. The posterior-conditioned forward pass is computed by the code, but its predicted latent is not used in the prediction or policy loss. Only its Gaussian parameters enter the posterior KL penalty. Thus the VJEPA-labelled baseline uses deterministic regression plus separate posterior regularization, rather than the variational expectation in Eq. (14). For a nonlinear predictor, evaluation at $h = 0$ need not equal the predictive mean marginalized over h.

Implementation provenance. The available repository history contains one version of the core training code. That initial release bundles the model, loss, training routine, and experiment configurations with all 53 compact result CSVs; their checksums match the release manifest. Later commits do not change these core files. This establishes their release association, not the exact code executed for each historical run: the original checkpoints, run logs, and run-level source identifiers were not recovered. We describe the available released implementation and retain its result labels, without claiming an independent reproduction of the full variational framework. The supplementary source includes the unmodified released code, configurations, checksums, and a small dependency check that confirms the separate prediction and posterior loss paths. Figure 4 uses that same training routine in an explicitly separate illustrative rerun.

Reproducibility and Artifact Availability. The project release contains training and analysis code, configurations, and 53 original compact result CSVs. The manuscript source package includes these summaries, revised plotting scripts, and the released implementation with its configurations and source checksums. Large per-sample and per-episode outputs, datasets, and checkpoints are excluded from the tracked release. The reported aggregates were checked against the compact outputs; the original aggregate studies were not independently rerun for this revision, and bootstrap intervals were not regenerated from raw episodes. Figure 4 instead uses an additional, explicitly separate illustrative PushT rerun. Its saved frames and policy outputs are included with the source. The release association of code and summaries was verified, but the exact source version executed for each original run could not be established (Appendix B.3). Shared-model dependence remains a limitation (Section 5).

B.4 Product-of-Experts BJEPA

The dynamics expert predicts diagonal Gaussian parameters $(\mu_{\text{dyn}}, \sigma^2_{\text{dyn}})$ from the current latent, action, task embedding, and auxiliary input fixed to $h = 0$. Its Gaussian NLL is evaluated at that fixed input; it is not averaged over posterior samples. The prior expert is an empirical diagonal Gaussian $(\mu_{\text{prior},\ell}, \sigma^2_{\text{prior},\ell})$ fitted for task ℓ on training target latents. It does not use z_{t+1} at inference. The combined prediction is

$$\tau_{\text{B}} = \sigma^{-2}_{\text{dyn}} + \sigma^{-2}_{\text{prior}}, \tag{3}$$

$$\sigma^2_{\text{B}} = \tau^{-1}_{\text{B}}, \tag{4}$$

$$\mu_{\text{B}} = \sigma^2_{\text{B}} \left(\sigma^{-2}_{\text{dyn}} \mu_{\text{dyn}} + \sigma^{-2}_{\text{prior}} \mu_{\text{prior}} \right). \tag{5}$$

All operations are elementwise. Log variances are clipped to $[-10, 5]$, variances are lower-bounded by 10^{-6} before inversion, and training aborts on non-finite precision, variance, mean, or loss.

B.5 Training Algorithm

Algorithm 1: Leakage-free training and evaluation of VJEPA/BJEPA variants

1. Split demonstration episodes into seed-specific training and validation partitions; fit latent/action normalization on training data only.
2. Train a Gaussian BC reference policy on normalized training latents. For each source seed, reserve the policies from the other two seeds as evaluators.
3. For BJEPA variants, fit the task-conditioned empirical Gaussian prior using normalized training latents only.
4. For each minibatch, form context (z_t, a_t, ℓ). Compute the posterior $q_\psi(h \mid z_t, a_t, z_{t+1}, \ell)$ for its KL penalty only.
5. Predict with $h = 0$ and no future-target input. VJEPA uses a residual point predictor; BJEPA combines its dynamics and task-prior Gaussians.
6. Compute the base loss: latent MSE for VJEPA or predictive Gaussian NLL for BJEPA. Add $\beta D_{\text{KL}}(q_\psi(h) \| \mathcal{N}(0, I))$.
7. For DE variants, add $\lambda_{\text{DE}} D_{\text{KL}}(\pi_{\text{ref}}(\cdot \mid z_{t+1}) \| \pi_{\text{ref}}(\cdot \mid \hat{z}_{t+1}))$.
8. Update with AdamW. Prediction losses train the predictor; posterior KL trains the posterior. The task embedding is shared. Encoder and policy stay frozen.
9. At evaluation, retain the target-free $h = 0$ prediction path and score with independently seeded evaluator policies.

B.6 Complete Four-Method Results

Table 9: Four-method results: mean ± standard error over 30 paired task/source/evaluator units. Lower is better; bold marks the lowest mean per shift and metric.

Shift	Method	Latent MSE	Decision KL	Action MSE
Clean	VJEPA	**0.001506 ± 0.000067**	0.3496 ± 0.0237	0.000734 ± 0.000130
	BJEPA	0.002045 ± 0.000145	0.4257 ± 0.0230	0.001000 ± 0.000159
	DE-VJEPA	0.001738 ± 0.000074	**0.2803 ± 0.0207**	**0.000668 ± 0.000112**
	DE-BJEPA	0.002051 ± 0.000149	0.3606 ± 0.0207	0.000932 ± 0.000149
Brightness	VJEPA	0.05121 ± 0.00059	308.5 ± 66.4	0.07941 ± 0.00573
	BJEPA	0.04878 ± 0.00074	215.5 ± 41.9	0.07248 ± 0.00534
	DE-VJEPA	0.05113 ± 0.00062	235.8 ± 52.8	0.07436 ± 0.00546
	DE-BJEPA	**0.04731 ± 0.00073**	**206.2 ± 38.4**	**0.07089 ± 0.00512**
Blur	VJEPA	0.2413 ± 0.0043	613.7 ± 95.0	0.1458 ± 0.0122
	BJEPA	0.2218 ± 0.0043	**463.1 ± 58.2**	0.1355 ± 0.0120
	DE-VJEPA	0.2440 ± 0.0045	539.0 ± 75.5	0.1445 ± 0.0120
	DE-BJEPA	**0.2130 ± 0.0048**	486.4 ± 63.8	**0.1344 ± 0.0119**
Desaturation	VJEPA	0.1983 ± 0.0046	**3335 ± 1081**	**0.3290 ± 0.0546**
	BJEPA	0.1964 ± 0.0044	3756 ± 1193	0.3344 ± 0.0553
	DE-VJEPA	0.2010 ± 0.0047	3546 ± 1135	0.3315 ± 0.0559
	DE-BJEPA	**0.1915 ± 0.0043**	3766 ± 1198	0.3308 ± 0.0542

B.7 Paired Statistical Comparisons

The following are the reported paired tests over the analysis units described above. Their p-values are nominal and inherit the dependence discussed in the main text.

Against VJEPA, DE-VJEPA's clean decision-KL change is -0.0693 (-19.8%; paired t-test $p = 6.5\times10^{-14}$; Wilcoxon $p = 1.9\times10^{-9}$), while clean latent MSE changes by $+0.000231$ ($+15.4\%$; $p = 2.2\times10^{-18}$). Under brightness, DE-BJEPA versus VJEPA changes latent MSE by -0.00390 (-7.6%, $p = 6.6\times10^{-12}$), decision KL by -102.3 (-33.2%, $p = 0.0044$), and action MSE by -0.00852 (-10.7%, $p = 0.00093$). Under blur, BJEPA versus VJEPA reduces decision KL by 150.6 (24.5%, $p = 0.0073$). Under desaturation, BJEPA increases decision KL by 421.3 (12.6%, $p = 0.024$), demonstrating that the extension is not uniformly robust.

DE-BJEPA improves clean decision KL relative to BJEPA by 0.0651 ($p = 4.2\times10^{-11}$), but remains worse than DE-VJEPA on clean decision KL by 0.0803 ($p = 4.8\times10^{-11}$). Under brightness, DE-BJEPA has lower means than DE-VJEPA for all three metrics, although the decision-KL difference does not reach 0.05 significance ($p = 0.089$). These comparisons motivate the narrow conclusion that Bayesian predictive structure helps under brightness and blur, while policy-KL regularization is most reliable on clean observations.

PushT cross-dataset comparison. On PushT, the paired t-test for decision KL gives nominal p-values below 0.05 for clean ($p = 9.3\times10^{-7}$), brightness ($p = 0.0060$), blur ($p = 0.014$), and desaturation ($p = 0.0012$). However, PushT has only six source/evaluator pairs, so the smallest attainable two-sided Wilcoxon value is 0.03125.

B.8 Metrics, Calibration, and Saved Artifacts

For each held-out transition we save $\|z_{t+1} - \hat{z}_{t+1}\|_2^2$, $D_{\mathrm{KL}}(\pi_{\mathrm{eval}}(\cdot \mid z_{t+1}) \| \pi_{\mathrm{eval}}(\cdot \mid \hat{z}_{t+1}))$, and $\|\mu_{\mathrm{eval}}(z_{t+1}) - \mu_{\mathrm{eval}}(\hat{z}_{t+1})\|_2^2$. BJEPA variants additionally save diagonal Gaussian NLL and mean predictive variance. Calibration is assessed by quantile-binning predictions according to variance and plotting bin-level variance against latent error and decision KL.

The raw core file is `results/vjepa_bjepa_de_comparison.csv`; aggregate estimates are in `results/vjepa_bjepa_de_comparison_summary.csv`; prespecified paired tests are in `results/vjepa_bjepa_de_comparison_paired.csv`. The associated table and plots are generated by:

```
python scripts/run_experiment.py \
  --config configs/core_vjepa_bjepa_de.yaml
python scripts/analyze_results.py \
  --config configs/core_vjepa_bjepa_de.yaml
```

B.9 Extended Baseline Matrix

The matched two-dataset study uses the following conditions:

1. **Persistence**: $\hat{z}_{t+1} = z_t$, with no trainable predictor.
2. **Linear dynamics**: a single residual linear layer conditioned on z_t, a_t, and task identity.
3. **No-action VJEPA**: the standard predictor with a_t replaced by zero, testing whether action conditioning contributes.
4. **Cosine JEPA**: latent cosine distance replaces latent MSE.
5. **VJEPA**: the released latent-MSE point predictor at $h = 0$, with a separately KL-regularized posterior branch.
6. **Action-MSE**: VJEPA plus direct policy-mean matching at weight 0.1.
7. **Random-policy KL**: VJEPA plus KL from a frozen random policy.
8. **Weak-policy KL**: VJEPA plus KL from a two-epoch BC policy.
9. **DE-VJEPA**: VJEPA plus strong-BC policy KL at weight 0.01.
10. **BJEPA**: Gaussian PoE prediction trained with latent NLL.
11. **DE-BJEPA**: BJEPA plus strong-BC policy KL at weight 0.01.

The raw and aggregate outputs are `results/cross_method_metaworld.csv`, `results/cross_method_metaworld_summary.csv`, `results/cross_method_pusht.csv`, and `results/cross_method_pusht_summary.csv`. Paired comparisons are stored in the corresponding `_paired.csv` files. The invalid pre-normalization PushT run is retained only for auditability under an `_invalid_action_scale` filename.

C Closed-Loop Evaluation Details

C.1 Candidate-Action Planning Algorithm

Algorithm 2: One-step closed-loop evaluation with an independent BC controller

1. **Inputs:** frozen encoder f, independent seed-7 BC policy π_{BC}, optional predictor g_θ, task ℓ, visual condition c, candidate count $N = 32$, noise scale $\sigma_a = 0.12$, and deviation weight $\alpha = 0.1$.
2. Reset MetaWorld with the prespecified environment seed. Render the RGB frame, apply condition c, and encode $z_t = f(o_t)$.
3. Query the BC distribution $p_t = \pi_{\mathrm{BC}}(\cdot \mid z_t, \ell)$ and set $a_t^{(1)} = \mu(p_t)$.
4. For `bc_only`, execute $a_t^{(1)}$ and skip to Step 8.
5. For a predictor-assisted method, sample $a_t^{(i)} = \mathrm{clip}(a_t^{(1)} + \epsilon_i, -1, 1)$ for $i = 2, \ldots, N$, where $\epsilon_i \sim \mathcal{N}(0, \sigma_a^2 I)$.
6. Predict one-step latents $\hat{z}_{t+1}^{(i)} = g_\theta(z_t, a_t^{(i)}, 0, \ell)$ with fixed auxiliary input $h = 0$.
7. Score each candidate by
$$S_i = H\Big[\pi_{\mathrm{BC}}(\cdot \mid \hat{z}_{t+1}^{(i)}, \ell)\Big] + \alpha \|a_t^{(i)} - a_t^{(1)}\|_2^2,$$
and execute $a_t^{(i^\star)}$ for $i^\star = \arg\min_i S_i$.
8. Observe reward, termination, success, and the next rendered frame. Encode the realized z_{t+1} and, for world-model methods, log online latent MSE, independent decision KL, and action-mean MSE between z_{t+1} and the selected prediction.
9. Repeat until success, environment termination, or 200 steps. Append the completed episode to the raw CSV immediately so interrupted runs can resume.

Figure 10: Closed-loop candidate-action planning and leakage-free evaluation.

Why this planner is diagnostic. The planner does not use reward, privileged state, or a learned value function. It tests whether a predictor that better preserves policy geometry makes local BC action selection more stable. Its failure to beat BC-only does not imply that the learned dynamics are useless for every planner; it shows that local entropy minimization is insufficient in the present setting.

C.2 Closed-Loop Configuration

Table 10: Complete closed-loop evaluation configuration.

Field	Value
Tasks	REACH-V3, BUTTON-PRESS-V3, DRAWER-OPEN-V3
Methods	BC-only, BC+VJEPA, BC+DE-VJEPA
Predictor seeds	42, 1000, 10000
Deployment/evaluation policy seed	7; distinct from predictor seeds
Visual conditions	clean, brightness factor 0.55, deterministic background tint
Episodes	20 per task–seed–condition–method cell
Total episodes	1,620
Maximum horizon	200 environment steps
Candidate planner	32 candidates; Gaussian action noise 0.12
Candidate score	predicted BC entropy + 0.1 action-deviation penalty
DE regularizer checkpoint	$\lambda_{\mathrm{DE}} = 0.01$
Encoder	frozen ImageNet ResNet-18, 512-dimensional latent
Action selection	deterministic policy and predictor means
Primary endpoint	episode success
Secondary endpoints	return, length, final goal distance, action norm, policy entropy, online latent MSE, decision KL, action MSE
Compute	NVIDIA RTX 4090; approximately 3.3 wall-clock hours including resume

C.3 Aggregate Closed-Loop Results

Table 11: Closed-loop MetaWorld success rate (%) with 95% episode-bootstrap confidence intervals. Each condition contains 180 episodes per method.

Method	Clean	Brightness	Background shift
BC only	**79.4** [73.9, 85.6]	**12.8** [8.3, 17.8]	28.9 [22.2, 35.6]
BC + VJEPA	56.7 [49.4, 63.9]	4.4 [1.7, 7.8]	30.6 [23.9, 37.2]
BC + DE-VJEPA	57.2 [50.0, 64.4]	7.2 [3.3, 11.1]	**31.1** [23.9, 38.3]

C.4 Task-Level Closed-Loop Results

Table 12: Task-level closed-loop success rates (%). Each cell contains 60 episodes: 20 episodes for each of three predictor seeds.

Task	Method	Clean	Brightness	Background shift
Reach	BC only	40.0	11.7	8.3
	BC + VJEPA	31.7	6.7	**20.0**
	BC + DE-VJEPA	35.0	10.0	16.7
Button press	BC only	**98.3**	**26.7**	**78.3**
	BC + VJEPA	85.0	6.7	71.7
	BC + DE-VJEPA	95.0	11.7	76.7
Drawer open	BC only	**100.0**	0.0	0.0
	BC + VJEPA	53.3	0.0	0.0
	BC + DE-VJEPA	41.7	0.0	0.0

The aggregate pattern is heterogeneous. DE-VJEPA improves over VJEPA on clean reach and button press, and under brightness for both tasks, but it reduces clean drawer-open success. No method solves shifted drawer open. Background shift produces the only aggregate condition where predictor-assisted methods slightly exceed BC-only, but the improvement is driven by reach and is not statistically significant.

C.5 Closed-Loop Statistical Tests

The prespecified paired unit is the task–predictor-seed–visual-condition success rate, yielding 27 units overall and nine per condition. DE-VJEPA versus VJEPA gives:

- overall success difference +1.30 points, standard error 1.78, paired t-test $p = 0.473$, bootstrap CI $[-2.41, 4.44]$;
- clean +0.56 points, $p = 0.915$, CI $[-10.00, 8.89]$;
- brightness +2.78 points, $p = 0.179$, CI $[0.00, 6.67]$;
- background shift +0.56 points, $p = 0.681$, CI $[-1.67, 2.78]$;
- overall return difference +10.14, $p = 0.363$, CI $[-10.24, 32.66]$.

DE-VJEPA versus BC-only gives an overall success difference of -8.52 points ($p = 0.076$, CI $[-17.96, -0.74]$), clean difference -22.22 points ($p = 0.081$), brightness difference -5.56 points ($p = 0.095$), and background difference +2.22 points ($p = 0.719$). The bootstrap intervals operate on the 27 paired aggregate units; episode-bootstrap intervals in Table 11 describe uncertainty in each raw success proportion. These two intervals answer different questions and should not be interchanged. The recovered analysis code uses a paired t-test on aggregate differences and a percentile bootstrap with 2,000 resamples of those differences. These procedures can give different threshold decisions. We retain both outputs and do not treat interval exclusion alone as a confirmed effect. The shared-model dependence discussed in the main text remains a limitation of this analysis.

C.6 Online Prediction Diagnostics

For the executed candidate, the evaluator encodes the realized next frame and computes online prediction metrics. Averaged over 180 episodes per condition, DE-VJEPA has lower latent MSE than VJEPA under clean (0.00498 versus 0.00560), brightness (0.00720 versus 0.00802), and background shift (0.01127 versus 0.01265). It also has lower action MSE in all three conditions. Decision KL is lower under clean (2.34 versus 3.46) and background shift (46.0 versus 55.9), but similar and very large under brightness (52.1 versus 51.4). Thus, the online diagnostics largely reproduce the offline ordering without producing a significant task-success gain.

C.7 Raw Rollout Data Dictionary

Each row of `results/closed_loop_rollout_raw.csv` is one completed episode. The columns are:

`method` BC-only or predictor-assisted controller.

`task` MetaWorld environment name.

`seed` Predictor-training seed; repeated for BC-only to preserve paired cells.

`policy_seed` Independent deployment-policy seed, fixed to 7.

`visual_condition` Clean, brightness, or background shift.

`episode_id` Episode index within a task–seed–condition cell.

`environment_seed` Deterministic reset seed shared by methods.

`planner_type` BC action or one-step candidate planner.

`num_candidate_actions`
: Candidate count, 32 for this study.

`lambda_de` Requested decision-regularizer checkpoint weight.

`success` Binary MetaWorld success indicator.

`return` Sum of environment rewards.

`episode_length`
: Executed environment steps.

`final_distance_to_goal`
: Final simulator-provided object-to-target distance when available.

`failure_reason`
: Success, timeout, termination, or no-success diagnostic.

`mean_action_norm`
: Mean Euclidean norm of executed actions.

`mean_policy_entropy`
: Mean BC Gaussian entropy.

`mean_prediction_latent_mse`
: Mean online next-latent error.

`mean_decision_kl`
: Mean online independent policy KL.

`mean_action_mse`
: Mean online policy-mean error.

D Supplementary Integration-Gap Diagnostics

D.1 Planner Ablation Protocol

The planner ablation preserves the tasks, predictor seeds, independent seed-7 BC controller, visual conditions, episode horizon, candidate count, and action noise of the original closed-loop evaluation. It changes only the candidate score. Candidate perturbations are regenerated deterministically from the matched environment seed and step index, so VJEPA and DE-VJEPA receive exactly the same candidate actions. The evaluated scores are:

$$S_{\mathrm{ent}}(a_i) = H[\pi_{\mathrm{BC}}(\cdot \mid \hat{z}_{t+1}^{(i)})] + \alpha \|a_i - a_{\mathrm{BC}}\|_2^2, \tag{6}$$

$$S_{\mathrm{KL}}(a_i) = D_{\mathrm{KL}}\Big(\pi_{\mathrm{BC}}(\cdot \mid z_t) \| \pi_{\mathrm{BC}}(\cdot \mid \hat{z}_{t+1}^{(i)})\Big) + \alpha \|a_i - a_{\mathrm{BC}}\|_2^2, \tag{7}$$

$$S_{\mu}(a_i) = \|\mu_{\mathrm{BC}}(\hat{z}_{t+1}^{(i)}) - a_{\mathrm{BC}}\|_2^2 + \alpha \|a_i - a_{\mathrm{BC}}\|_2^2. \tag{8}$$

A random-candidate control samples the same candidate set but selects an index uniformly. BC-only is evaluated once per matched cell. This produces 4,860 episodes:

$$540 \text{ BC episodes} + 4 \times 2 \times 540 \text{ predictor–score episodes}.$$

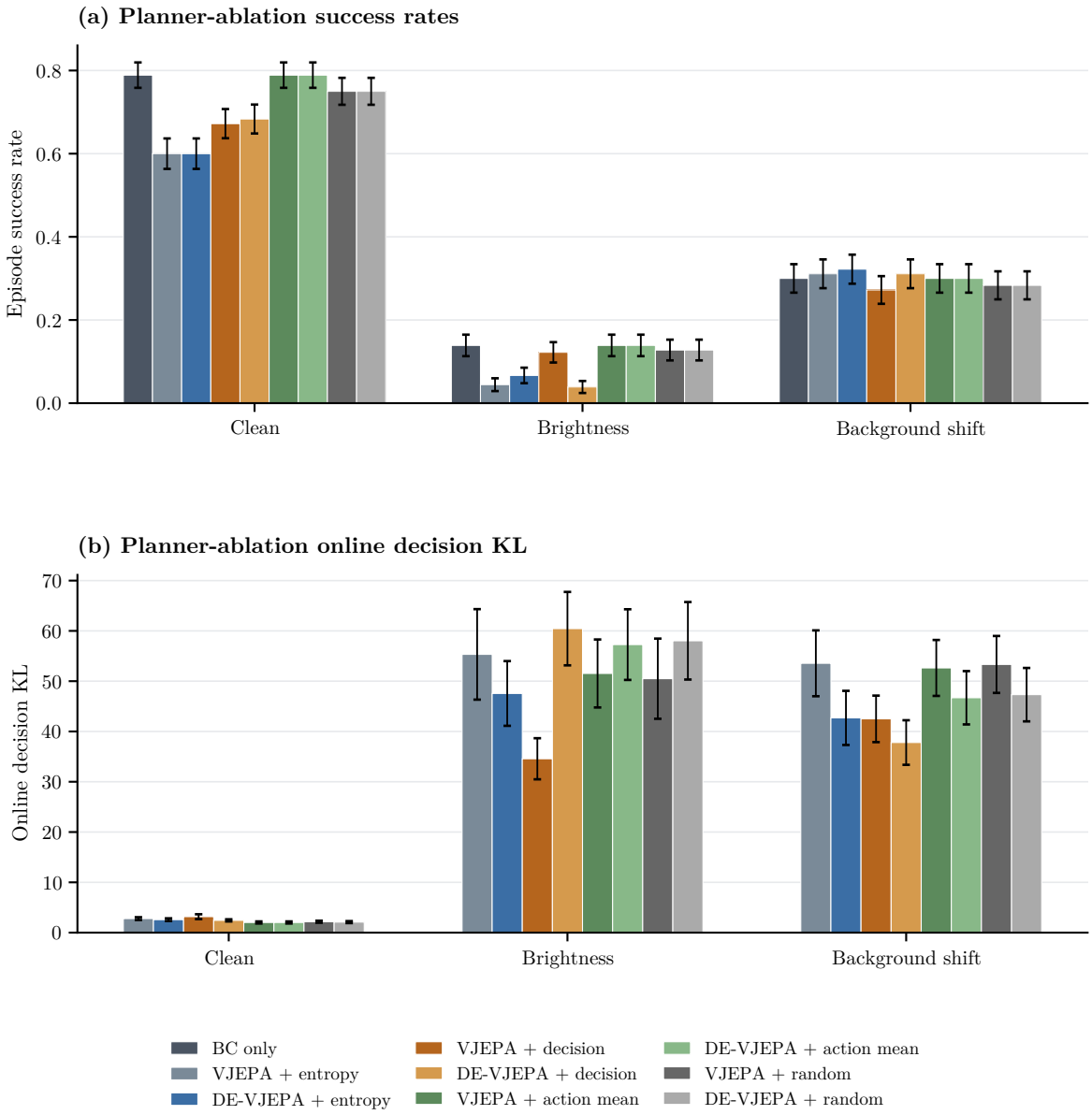


Figure 11: Planner-ablation success (top) and online decision KL (bottom), redrawn from the original compact results with unchanged means and standard errors. The shared legend applies to both panels. Direct BC has no predicted-latent decision KL. Improved online decision metrics do not imply improved task success. Action-mean consistency stays close to the BC action and reproduces BC-only success.

Table 13: Planner-ablation success rates (%). Each method–score–condition cell contains 180 episodes.

Method	Planner score	Clean	Brightness	Background	Overall
BC only	Direct BC mean	78.89	13.89	30.00	**40.93**
VJEPA	Entropy	60.00	4.44	31.11	31.85
DE-VJEPA	Entropy	60.00	6.67	32.22	32.96
VJEPA	Decision consistency	67.22	12.22	27.22	35.56
DE-VJEPA	Decision consistency	68.33	3.89	31.11	34.44
VJEPA	Action-mean consistency	78.89	13.89	30.00	**40.93**
DE-VJEPA	Action-mean consistency	78.89	13.89	30.00	**40.93**
VJEPA	Random candidate	75.00	12.78	28.33	38.70
DE-VJEPA	Random candidate	75.00	12.78	28.33	38.70

D.2 Planner Ablation Statistical Results

The paired unit is task–predictor-seed–visual-condition, yielding 27 units overall and nine within each condition. DE-VJEPA versus VJEPA gives:

- entropy score: +1.11 success points, $p = 0.680$, bootstrap CI $[-4.07, 6.30]$;
- decision-consistency score: -1.11 points, $p = 0.597$, CI $[-5.19, 2.78]$;
- action-mean consistency: exactly 0.00 points;
- random-candidate control: exactly 0.00 points because candidate generation and random selection are paired and do not depend on the predictor.

No planner score establishes a DE-VJEPA success advantage over VJEPA.

Relative to BC-only, entropy scoring reduces VJEPA success by 9.07 points ($p = 0.0215$) and DE-VJEPA success by 7.96 points ($p = 0.0564$). Decision-consistency scoring reduces VJEPA success by 5.37 points ($p = 0.0341$) and DE-VJEPA success by 6.48 points ($p = 0.00737$). Random candidate selection reduces success by 2.22 points ($p = 0.0202$). Action-mean consistency has zero success difference from BC-only.

Action deviation and success. The mean root-mean-square action deviation from BC is approximately 0.12–0.15 for entropy and decision-consistency scoring, 0.107 for the random control, and less than 6×10^{-4} for action-mean consistency. The score that matches BC success therefore also keeps the executed actions closest to the BC mean. This association helps explain the observed ranking, but does not isolate the effect of action deviation from the other consequences of each score.

Condition-specific behavior. Entropy-scored DE-VJEPA is directionally better than entropy-scored VJEPA under brightness (6.67% versus 4.44%) and background shift (32.22% versus 31.11%), but neither difference is significant. Decision consistency performs poorly for DE-VJEPA under brightness (3.89%) despite improved predicted decision geometry elsewhere. This is evidence that minimizing local distribution change can favor conservative or dynamically unhelpful actions.

D.3 Autoregressive Multi-Step Diagnostic

To check whether the offline advantage survives repeated prediction, trained one-step VJEPA and DE-VJEPA predictors are rolled autoregressively over held-out MetaWorld demonstration sequences. The predictor receives the recorded action sequence but never receives future latents. Horizons $K \in \{1, 3, 5\}$ are scored with the independent seed-7 BC evaluator. The final raw file contains 63,846 method–sample–horizon rows.

DE-VJEPA reduces decision KL relative to VJEPA by approximately 20.5% at $K = 1$, 12.8% at $K = 3$, and 13.7% at $K = 5$. Its mean decision-error slope is also lower (0.255 versus 0.288 per prediction step).

Table 14: Multi-step diagnostic averaged across reach, button press, and drawer open. Lower is better.

Method	K	Latent MSE	Decision KL	Action MSE
VJEPA	1	**0.001336**	0.3778	0.000289
DE-VJEPA	1	0.001545	**0.3004**	**0.000267**
VJEPA	3	**0.003233**	0.9050	**0.000688**
DE-VJEPA	3	0.003727	**0.7888**	0.000694
VJEPA	5	**0.005170**	1.5311	**0.001124**
DE-VJEPA	5	0.005886	**1.3214**	0.001138

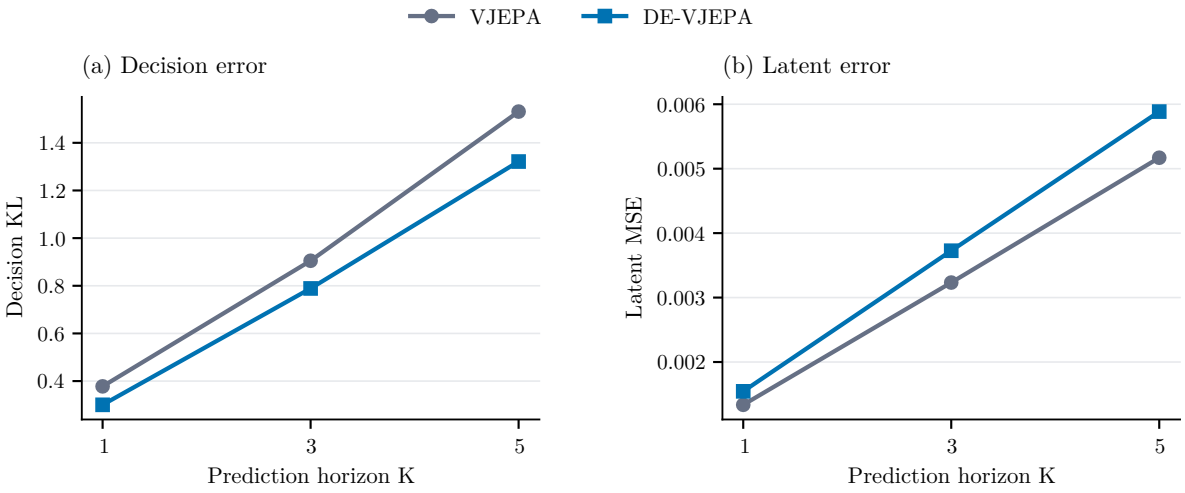


Figure 12: Autoregressive decision KL (left) and latent MSE (right), redrawn from the original task-level summary CSV. The plot and Table 14 both average the three tasks equally. DE-VJEPA maintains lower decision KL through horizon five while retaining higher latent error. No error bars are inferred from the reported means.

In contrast, DE-VJEPA latent MSE is higher at every horizon, with latent-error slopes 0.001085 versus 0.000958. The decision advantage therefore weakens from its one-step maximum but does not disappear by $K = 5$.

The original plotting script weighted task means by their sample counts, whereas its table-generation code averaged the tasks equally. This explains the numerical difference between the earlier plot and table. The revised plot uses equal task weights to match the table and the percentages above.

D.4 Diagnosis of the Integration Gap

The diagnostics establish several observations within the tested setting:

1. DE-VJEPA preserves lower decision KL through five prediction steps.
2. DE-VJEPA does not significantly beat VJEPA under any tested local candidate score.
3. Entropy and decision-consistency scores underperform direct BC.
4. Random perturbations also reduce success relative to direct BC.
5. Action-mean consistency matches BC success while keeping action deviations negligible.

The decision-KL advantage survives repeated prediction along demonstration actions, but none of the tested local scores turns it into a reliable success gain. The planner may encounter states and actions outside those evaluated offline, so this result does not rule out compounding model error during control. It also leaves open whether a more task-relevant scoring function would help.

Appendix E tests one such score using normalized demonstration time. Other possible experiments include reward- or value-based scores, uncertainty-aware planning, and multi-step predictor training. Each would need a matched evaluation to determine which part of the system limits control performance.

E Supplementary Progress-Aware Planning Results

E.1 Progress-Head Validation

The available MetaWorld demonstrations contain successful trajectories but do not store reward-to-go or a task-specific distance target. We therefore train a task-conditioned *progress head*, not a value function, using $y_t = t/(T-1)$. The MLP has two 512-unit hidden layers and is trained independently with seeds 42, 1000, and 10000. Table 15 reports validation statistics averaged over those seeds.

Table 15: Validation of the 512-unit trajectory-progress head. Correlation is with normalized time within a successful demonstration.

Task	MSE ↓	Pearson ↑	Spearman ↑
All tasks	0.000356	0.99794	0.99796
Reach	0.000990	0.99437	0.99431
Button press	0.000200	0.99897	0.99911
Drawer open	0.000078	0.99956	0.99956

The head easily reconstructs trajectory position. This validates the implementation and provides a smooth planning score, but it does not establish that the score measures causal task progress. In particular, a planner can increase predicted trajectory time while moving to an off-demonstration latent.

E.2 One-Step Progress Planning

The one-step study evaluates 4,860 episodes over three tasks, three predictor seeds, three visual conditions, four action-deviation weights, and matched VJEPA/DE-VJEPA predictors, with a shared BC-only control.

At $\alpha = 0.1$, where actions remain close to BC, VJEPA and DE-VJEPA obtain 40.56% and 40.93% aggregate success, respectively. Their condition-level success rates are 77.78/78.89% on clean observations, 13.33/13.33% under brightness, and 30.56/30.56% under background shift. The paired aggregate difference is +0.37 percentage points ($p = 0.161$).

Smaller penalties make the planner more active but less reliable. At $\alpha = 0.01$, DE-VJEPA improves over VJEPA under brightness by 9.44 points (24.44% versus 15.00%), but the paired result is not significant ($p = 0.231$), and clean success falls to 41.11%. At $\alpha = 0.3$, both predictor planners nearly reproduce BC. The one-step result therefore does not establish successful model-based control.

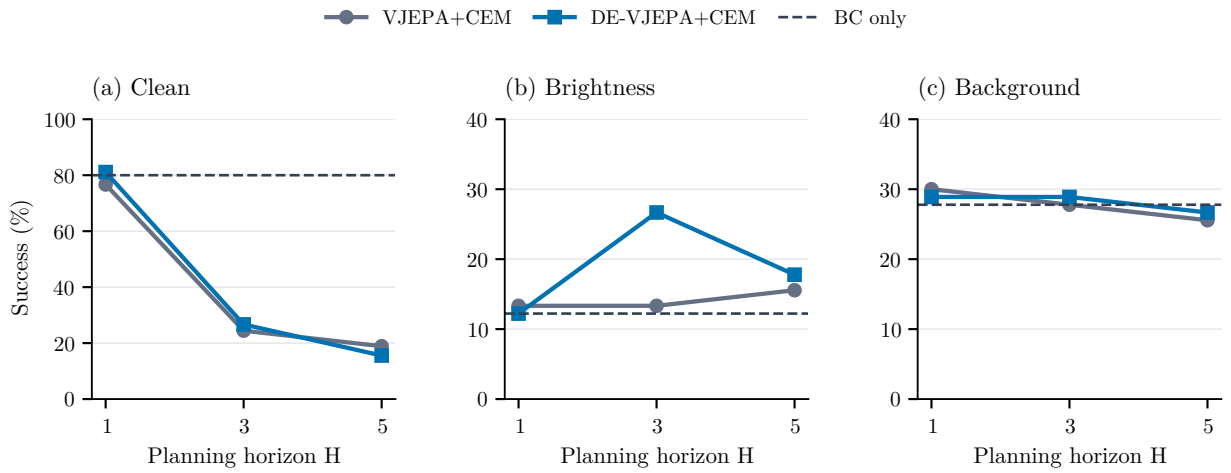


Figure 13: Progress-aware CEM success by planning horizon at $\alpha = 0.1$, shown separately for each visual condition. Points are the reported means in Table 16; dashed lines show the corresponding BC-only means. The y-axis ranges differ between clean and shifted conditions. No error bars are reconstructed because their numerical inputs were not recovered. Longer horizons reduce clean success.

E.3 Progress-Aware CEM/MPC

CEM uses population 128, three refinement iterations, horizons $H \in \{1, 3, 5\}$, and action penalties $\alpha \in \{0.01, 0.1\}$. The complete study contains 3,510 episodes. Table 16 reports the more stable $\alpha = 0.1$ setting.

Table 16: Progress-aware CEM success rates (%) at $\alpha = 0.1$. Each method–horizon–condition cell contains 90 episodes.

Method	H	Clean	Brightness	Background	Overall
BC only	–	80.00	12.22	27.78	40.00
VJEPA+CEM	1	76.67	13.33	30.00	40.00
DE-VJEPA+CEM	1	81.11	12.22	28.89	40.74
VJEPA+CEM	3	24.44	13.33	27.78	21.85
DE-VJEPA+CEM	3	26.67	26.67	28.89	27.41
VJEPA+CEM	5	18.89	15.56	25.56	20.00
DE-VJEPA+CEM	5	15.56	17.78	26.67	20.00

At $H = 1$, the two model-based planners are close to BC overall, and the DE-VJEPA–VJEPA difference is +0.74 points ($p = 0.537$). At $H = 3$, DE-VJEPA exceeds VJEPA by 5.56 points overall ($p = 0.113$, paired 95% bootstrap CI $[-0.37, 12.59]$) and by 13.33 points under brightness ($p = 0.073$, CI $[2.22, 24.44]$). The paired p-values do not cross the prespecified significance threshold. The brightness bootstrap interval

excludes zero, so it gives a different threshold decision from the paired test. The recovered code confirms that the interval uses a percentile bootstrap with 2,000 resamples, while the p-value comes from a paired t-test. Both operate on nine task–seed differences for brightness. We report both outputs and regard the control benefit as inconclusive. At $H = 5$, aggregate success is identical (20.00%).

E.4 Interpretation and Future Work

The progress-aware planners show the same unresolved transfer problem as the local-score planners. The mean advantage for DE-VJEPA at horizon three is worth investigating, but clean success declines substantially as the planning horizon grows. The present results cannot tell whether the horizon-three difference reflects better predictions for planning or sampling variation.

A follow-up could train the predictors at $K = 3$ and repeat the matched $H = 3$ CEM comparison with more independent training seeds. That would test whether matching the training and planning horizons helps. Repeating the offline study with stronger encoders would address a separate question: whether the policy-KL result extends beyond the current representation.

F Unevaluated Objective Variants

The following variants are proposals, not evaluated methods. The released DE-VJEPA configurations use the hybrid loss in Eq. (2). Rolling out a one-step-trained predictor does not test a multi-step training objective.

F.1 Reliability-Aware Policy Geometry

A reference policy can be unreliable away from its demonstration data (Guo et al., 2017). One proposal ignores discrepancies within a state-dependent tolerance:

$$d_{\pi,\epsilon}(\hat{z}, z) = \left[D_{\mathrm{KL}}(\pi_{\mathrm{ref}}(\cdot \mid z, \ell) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid \hat{z}, \ell)) - \epsilon(\hat{z}, z) \right]_+ , \tag{9}$$

where $[x]_+ = \max(x, 0)$. The tolerance could depend on an uncertainty score:

$$\epsilon(\hat{z}, z) = \epsilon_0 + c\,(u(\hat{z}) + u(z)) , \tag{10}$$

Here, $u(z)$ could be policy entropy, ensemble disagreement, or distance to the demonstration manifold. Neither formula guarantees calibrated uncertainty or more reliable gradients; both require validation.

F.2 Pure Decision-Equivalence Objective

One proposal removes the latent prediction term and also replaces the released $h = 0$ path with posterior-sampled prediction:

$$\begin{aligned} \mathcal{L}_{\text{DE-VJEPA}} = \mathbb{E}_{q_\psi} \left[d_{\pi,\epsilon}(\hat{z}_{t+k}, z_{t+k}) \right] \\ + \beta D_{\mathrm{KL}}(q_\psi(h \mid z_t, a_{t:t+k}, z_{t+k}) \,\|\, p(h)) . \end{aligned} \tag{11}$$

where h is the VJEPA auxiliary latent and

$$\hat{z}_{t+k} \sim p_\theta(\cdot \mid z_t, a_{t:t+k}, h).$$

The encoder and reference policy remain frozen. Unlike the evaluated hybrid, this loss has no direct constraint on latent prediction error.

F.3 Multi-Step Decision Equivalence

A multi-step training loss could compare policy outputs along a rollout:

$$d_\pi^{(K)} = \sum_{j=1}^{K} \gamma^{j-1} D_{\mathrm{KL}}(\pi_{\mathrm{ref}}(\cdot \mid z_{t+j}, \ell) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid \hat{z}_{t+j}, \ell)) \,. \tag{12}$$

Its effect on planning remains untested. A value-based alternative is

$$d_V(\hat{z}, z) = |V^{\pi_{\mathrm{ref}}}(\hat{z}, \ell) - V^{\pi_{\mathrm{ref}}}(z, \ell)| \,, \tag{13}$$

which requires an estimate of long-run value under the reference policy. The progress heads in Appendix E predict normalized demonstration time and do not validate this value-based loss.

G Policy Equivalence: Definitions and Proof

G.1 Policy-Equivalence Classes

The decision term distinguishes latents through their effect on a fixed policy. If two latents produce the same action distribution, this term cannot separate them. The resulting equivalence classes describe the information visible to the loss; they do not guarantee that training recovers the true latent or a state representation sufficient for optimal control.

Definition 1 (Policy equivalence)**.** *For a fixed task context ℓ, define*

$$z \sim_\pi z' \quad \Longleftrightarrow \quad \pi_{\mathrm{ref}}(\cdot \mid z, \ell) = \pi_{\mathrm{ref}}(\cdot \mid z', \ell).$$

The equivalence class of z is

$$[z]_\pi = \{z' : z' \sim_\pi z\} \,.$$

Proposition 1 (Zero decision loss)**.** *For any two latents z and $\hat{z}$,*

$$d_\pi(\hat{z}, z) = 0 \quad \Longleftrightarrow \quad \hat{z} \in [z]_\pi.$$

Thus, the decision term assigns zero error to exactly the latents that the reference policy treats as equivalent.

Proof. KL divergence is zero if and only if its two distributions agree almost everywhere. Applying this fact to Eq. (1) gives the definition of $\hat{z} \in [z]_\pi$. □

Corollary 1 (Injective policy as a special case)**.** *If $\pi_{\mathrm{ref}}(\cdot \mid z, \ell)$ is injective in z over the support of the predictor, then*

$$d_\pi(\hat{z}, z) = 0 \quad \Longleftrightarrow \quad \hat{z} = z.$$

This is a special case of the zero-loss condition, not an assumption about the policies used in our experiments.

Remark 1 (Scope of the equivalence)**.** *Different latents can legitimately induce the same action distribution, for example when they differ only in task-irrelevant background information. Proposition 1 describes the decision term alone. The hybrid objective still penalizes latent prediction error within an equivalence class, so the proposition does not imply that a trained DE-VJEPA predictor must collapse those latents.*

G.2 Limitations of the Reference Policy

The reference policy can also miss a distinction that matters for the task. For example, suppose two latents satisfy

$$\pi_{\mathrm{ref}}(\cdot \mid z, \ell) = \pi_{\mathrm{ref}}(\cdot \mid z', \ell)$$

but the optimal policy satisfies

$$\pi^*(\cdot \mid z, \ell) \neq \pi^*(\cdot \mid z', \ell),$$

Then the decision term cannot distinguish these latents, even though an optimal policy would. The latent term may still separate them, but the added policy penalty supplies no corrective signal. This limitation motivates the reference-policy quality ablation.

H Extended Background and Related Work

This appendix retains the full variational formulation and broader literature context. The evaluated prediction path and its scope are defined in Section 3.

H.1 Full Variational JEPA Formulation

Variational JEPA (VJEPA) represents this uncertainty by learning a predictive distribution over future latents (Huang, 2026). Conditioned on the same context and actions, the model can assign probability to different possible future representations.

In an action-conditioned formulation, an auxiliary latent variable h allows the predictor to represent variation in the future. Integrating over its prior $p(h)$ gives

$$p_\theta(z_{t+k} \mid z_t, a_{t:t+k}) = \int p_\theta(z_{t+k} \mid z_t, a_{t:t+k}, h)\, p(h)\, dh.$$

Training uses a variational posterior $q_\psi(h \mid z_t, a_{t:t+k}, z_{t+k})$, which can inspect the recorded future target to infer h. The objective combines a prediction term with a penalty that keeps this posterior close to the prior:

$$\mathcal{L}_{\text{VJEPA}} = \mathbb{E}_{q_\psi}[-\log p_\theta(z_{t+k} \mid \cdot)] + \beta\, D_{\text{KL}}(q_\psi \,\|\, p(h))\,. \tag{14}$$

The negative log-likelihood term rewards assigning probability to the observed future representation. The KL term compares distributions over the auxiliary variable h, with β controlling its weight. This differs from the action-distribution KL in our added loss. At evaluation, the predictor cannot use the target-conditioned posterior and must instead predict using the prior. The objective therefore models future uncertainty and regularizes the auxiliary variable. Neither term explicitly compares what a controller would do at the predicted and observed future states. Identifying which prediction errors matter for behavior is a separate choice.

Equation (14) describes the variational framework, not the objective implemented in the released experiments. Section 3 specifies their deterministic prediction path and separate posterior penalty.

H.2 Extended Related Work

Latent predictive representation learning. Image JEPA and video V-JEPA study representation learning through feature prediction (LeCun, 2022; Assran et al., 2023; Bardes et al., 2024); further work connects latent prediction to action-conditioned world modeling and planning (Garrido et al., 2024; Assran et al., 2025; Destrade et al., 2026). Video V-JEPA and Variational JEPA are distinct formulations. The latter is the probabilistic framework of Huang (2026); the archived result label *VJEPA* refers to the restricted implementation in Section 3, not an independently verified reproduction of that full framework. Other objectives use reconstruction, as in masked autoencoders (He et al., 2022), or contrastive prediction (Oord et al., 2018). Our focus is the criterion used to score a latent prediction for action choice.

World models for control. Classical latent world models learn compact dynamics for planning or policy optimization (Ha and Schmidhuber, 2018; Hafner et al., 2019; 2020; 2021; 2023). More recent scalable model-based agents such as MuZero, TD-MPC, and TD-MPC2 optimize behavior through latent prediction, value estimation, or model-predictive control rather than direct pixel reconstruction (Schrittwieser et al., 2020; Hansen et al., 2022; 2024). Generative systems such as UniSim, GAIA-1, and Genie offer other ways to

model future observations (Yang et al., 2023; Hu et al., 2023; Bruce et al., 2024); Hou et al. (2026) survey world models for robot learning. We focus on a narrower component of these systems: the loss used to train a latent predictor.

Robot policies and imitation learning. Imitation and visuomotor learning provide practical policies from demonstrations, including diffusion policies, robotics transformers, vision-language-action models, and action-chunking transformers (Ross et al., 2011; Chi et al., 2023; Brohan et al., 2022; Zitkovich et al., 2023; Zhao et al., 2023). We use a simpler Gaussian behavior-cloned (BC) policy to obtain a tractable KL loss. Because imitation policies can suffer from compounding errors and distribution shift (Ross et al., 2011), we test reference-policy quality and use evaluation policies trained with other seeds. RHyME and X-Sim address related alignment problems under mismatched demonstrations and embodiments (Kedia et al., 2024; Dan et al., 2025).

Decision-aware distances and calibration. Our policy-dependent loss is related to bisimulation metrics and invariant representations for control (Ferns et al., 2004; 2011; Zhang et al., 2021), and to the use of task-dependent distances in metric learning (Weinberger and Saul, 2009). It also inherits the reference policy's calibration errors: accurate neural predictions need not carry reliable probabilities (Guo et al., 2017). Appendix F describes an uncertainty-band proposal motivated by this concern; the reported core experiments do not test it.